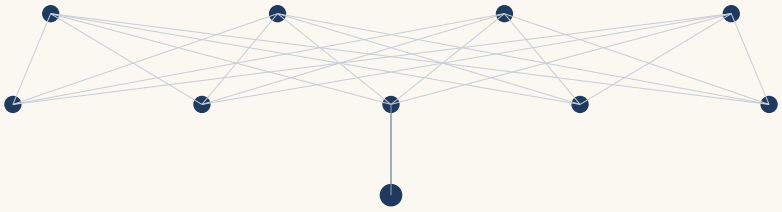


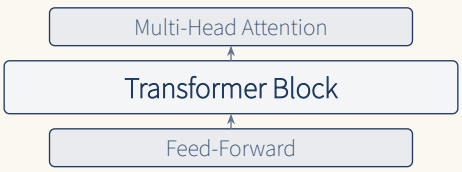
Make LLM
basic

From tokenizer to transformer

Large language model created from scratch

dirmich

Highmaru Press



Contents

Glossary ii

Preface vi

1 LLMWhat is 1
- 1.1 What is a language model? 1
- 1.2 intuition: How to think about language models 2
- 1.3 LLMDevelopment history of 2
  - 1.3.1 Statistical Language Models (1990s – 2000s) 2
  - 1.3.2 Neural Network Language Models (2003 – 2017) 3
  - 1.3.3 Rise of the Transformers (2017) 3
  - 1.3.4 GPT Series and Scaling Laws (2018) – today) 3
  - 1.3.5 The Rise of Sort (2022) – today) 4
- 1.4 LLMMain applications of 4
  - 1.4.1 Natural language processing traditional tasks 4
  - 1.4.2 Create task 5
  - 1.4.3 bailiwick 5
  - 1.4.4 multimodal 5
- 1.5 Why you should make it yourself 5
- 1.6 The model this book will create 6
- 1.7 Learning Roadmap 6
- 1.8 summation 6
- 1.9 practice problems 7

2 Development environment settings 8
- 2.1 tools needed 8
- 2.2 PyTorch installation 8
- 2.3 repository structure 9
- 2.4 first example: Dealing with tensors 9
- 2.5 Ensure reproducibility 10
- 2.6 Run unit tests 10
- 2.7 summation 10

3 Creating a tokenizer 11
- 3.1 Why do you need a tokenizer? 11
  - 3.1.1 Comparison of three approaches 11
- 3.2 special token 11

3.3 base class . . . 12
3.4 BPE tokenizer . . . 13
3.4.1 Algorithm Intuition . . . 13
3.4.2 Why Byte Unit?? . . . 14
3.4.3 avatar: learning . . . 14
3.4.4 encoding . . . 15
3.4.5 Preprocessing: White space processing . . . 16
3.4.6 decoding: byte restoration . . . 16
3.5 Unigram tokenizer . . . 17
3.5.1 intuition . . . 17
3.5.2 algorithm . . . 17
3.5.3 Viterbi decoding . . . 17
3.6 BPE vs Unigram comparison . . . 18
3.7 Tokenizer Test . . . 18
3.8 practice: Training tokenizer with real corpora . . . 19
3.9 summation . . . 20
3.10 practice problems . . . 20

4 Embedding and Positional Encoding 21
4.1 Why do you need embedding? . . . 21
4.2 Mathematical definition of embedding . . . 21
4.3 Token Embedding avatar . . . 22
4.3.1 padding_idxrole of . . . 22
4.4 Positional Encoding . . . 23
4.4.1 intuition: Why Order Matters? . . . 23
4.4.2 Sinusoidal Position encoding (Vaswani et al. 2017) . . . 23
4.4.3 Learnable location embeddings (GPT-2 styles) . . . 24
4.5 Embedding composition . . . 24
4.5.1 Why do you need scaling?? . . . 25
4.6 Layer Normalization introduction . . . 25
4.7 test . . . 26
4.8 practice: Embedding visualization . . . 26
4.9 summation . . . 26
4.10 practice problems . . . 27

5 attention mechanism 28
5.1 intuition: What is Attention? . . . 28
5.1.1 intuitive analogy: library search . . . 28
5.2 Q, K, VWhere does it come from? . . . 29
5.3 scaling: why – Divide by? . . . 29
5.4 Scaled Dot-Product Attention avatar . . . 29
5.5 causal mask (Causal Mask) . . . 30
5.5.1 why – impression? . . . 30
5.6 Multi-Head Attention . . . 31
5.6.1 idea . . . 31

5.6.2 Why split the head? . . . 31
5.6.3 avatar . . . 32
5.7 causality test . . . 32
5.8 Self-Attention vs Cross-Attention . . . 33
5.8.1 How is cross-attention different?? . . . 33
5.9 Attention calculation volume analysis . . . 33
5.10 practice: Attention weight visualization . . . 33
5.11 summation . . . 34
5.12 practice problems . . . 34

6 Assembling the transformer blocks 36
6.1 Structure of transformer block . . . 36
6.2 Feed-Forward Network (FFN) . . . 37
6.2.1 intuition: why expand-Is it shrinking?? . . . 37
6.3 GELU vs ReLU . . . 37
6.3.1 why GELUIs it better?? . . . 38
6.4 Connect residuals (Residual Connection) . . . 38
6.4.1 intuition: Why residuals help? . . . 38
6.5 Pre-LN vs Post-LN . . . 39
6.5.1 Post-LN (Vaswani text) . . . 39
6.5.2 Pre-LN (GPT-2 styles) . . . 39
6.5.3 why Pre-LNIs this more stable?? . . . 39
6.6 Residual scaling . . . 40
6.6.1 why – impression? . . . 40
6.7 block test . . . 40
6.8 Computation amount of transformer block . . . 41
6.9 summation . . . 41
6.10 practice problems . . . 41

7 GPT Model completion and learning loop 42
7.1 GPT model structure . . . 42
7.2 weighted tie (Weight Tying) . . . 42
7.3 Forward Pass . . . 43
7.4 dataset: sliding windows . . . 44
7.5 loss function: Cross-Entropy . . . 45
7.5.1 Label Smoothing (select) . . . 45
7.6 optimizer: AdamW . . . 45
7.7 learning rate scheduler . . . 46
7.7.1 why warmupIs this necessary?? . . . 46
7.8 gradient clipping . . . 47
7.9 trainer . . . 47
7.10 Perplexity . . . 47
7.11 Save checkpoint/load . . . 48
7.12 summation . . . 48
7.13 practice problems . . . 49

8 Text creation — Sampling Strategy and Decoding 50
8.1 Principle of autoregressive generation . . . 50
8.2 Greedy sampling . . . 50
8.3 Temperature sampling . . . 51
8.4 Top-K sampling . . . 51
8.4.1 Top-Kintuition . . . 52
8.5 Top-P (Nucleus) Sampling . . . 52
8.5.1 Top-K vs Top-P . . . 53
8.6 Sampling Strategy Comparison . . . 53
8.7 Implementing the creation function . . . 53
8.8 Context length limit . . . 54
8.9 sampler test . . . 54
8.10 practice: Comparison of various sampling . . . 55
8.11 Advanced decoding techniques (introduction) . . . 55
8.11.1 Beam Search . . . 55
8.11.2 Speculative Decoding . . . 55
8.11.3 Contrastive Search . . . 55
8.12 summation . . . 55
8.13 practice problems . . . 56

9 Clean up and next steps 57
9.1 Volume 1 Summary . . . 57
9.1.1 implemented . . . 57
9.1.2 test . . . 57
9.1.3 What the reader has . . . 57
9.2 Limitations of Volume 1 . . . 58
9.3 Covered in Volume 2 . . . 58
9.3.1 Distributed Learning (2 – Chapter 4) . . . 58
9.3.2 Modern Architecture (5 – Chapter 6) . . . 58
9.3.3 Quantization and inference optimization (Chapter 7) . . . 58
9.3.4 Alignment and fine tuning (Chapter 8) . . . 59
9.3.5 Data Pipeline (Chapter 9) . . . 59
9.4 Recommended Learning Path . . . 59
9.5 finally . . . 59

A Math Basics Supplement 60
A.1 vectors and matrices . . . 60
A.2 Softmax . . . 60
A.3 Cross-Entropy . . . 60
A.4 chain rule and Backpropagation . . . 61
A.5 matrix differentiation . . . 61
A.6 normal distribution . . . 61
A.7 Information Theory Basics . . . 61
A.8 Optimization Basics . . . 62

B Full list of codes 63

B.1 project structure . . . 63
B.2 run test . . . 63
B.3 Quickstart example . . . 64
B.4 By module API summation . . . 65
B.5 Setting data class . . . 65
B.6 license . . . 65

C Frequently Asked Questions and Pitfalls 66
C.1 environmental related . . . 66
C.1.1 Q: PyTorch After installation CUDAis not recognized . . . 66
C.1.2 Q: Learning is too slow . . . 66
C.2 Tokenizer related . . . 66
C.2.1 Q: BPEThe tokenizer learned with “hi”It tokenizes strangely. . . . 66
C.2.2 Q: encode/decode Round trip fails . . . 67
C.2.3 Q: special token IDWhat happens if changes . . . 67
C.3 model related . . . 67
C.3.1 Q: Attention output NaNThis comes out . . . 67
C.3.2 Q: The model repeats the same word . . . 67
C.3.3 Q: An error occurs if the context length is exceeded. . . . 67
C.4 learning related . . . 68
C.4.1 Q: Losses do not decrease . . . 68
C.4.2 Q: Loss occurs in the early stages of learning. . . . 68
C.4.3 Q: Loading a checkpoint gives different results . . . 68
C.5 Distributed Learning (Preview of Volume 2) . . . 68
C.5.1 Q: DDPIf you learn with , the speed is rather slow. . . . 68
C.5.2 Q: FSDPat OOMThis is me . . . 68
C.6 Debugging Tips . . . 69
C.6.1 Verify step by step . . . 69
C.6.2 shapeAlways print . . . 69
C.6.3 Start with small input . . . 69
C.7 Performance Optimization Tips . . . 69
C.7.1 Use mixed precision . . . 69
C.7.2 torch.compile . . . 69
C.7.3 profiling . . . 70

D Practice Problem Solutions 71
D.1 Chapter 1 Answers . . . 71
D.2 Chapter 3 Answers . . . 71
D.3 Chapter 4 Answers . . . 72
D.4 Chapter 5 Answers . . . 73
D.5 Chapter 6 Answers . . . 73
D.6 Chapter 7 Answers . . . 74
D.7 Chapter 8 Answers . . . 74

E Advanced topic preview 75
E.1 distributed learning . . . 75

E.2 modern architecture 75
E.3 Quantization and inference optimization 76
E.4 Alignment and fine tuning 76
E.5 data pipeline 76
E.6 Volume 2 Study Preparation 76
E.7 Recommended Learning Path 76

Glossary

This glossary organizes the main terms used in this book in alphabetical order. Each item is organized with a short definition and the corresponding chapter number for easy reference by readers. When you first read the book, skim it lightly, and if you come across a term you don't know while reading the text, come back and refer to it.

Activation (activation value)
: The value output after passing through each layer of the neural network. This refers to the output tensor of the layer and becomes the input to the next layer.reference: Chapter 4

AdamW
: An optimization algorithm that combines the Adam optimizer with weight decay. The de facto standard for modern LLM learning.reference: Chapter 7

Attention
: A mechanism by which each position in a sequence learns how much attention to pay to every other position. The core of Transformers.reference: Chapter 5

Backpropagation
: An algorithm that calculates the gradient by differentiating the loss of a neural network by each parameter. Based on the chain rule.reference: Chapter 7

BPE (Byte Pair Encoding)
: Tokenizer algorithm starts byte by byte and merges the most frequently occurring pairs. Used by GPT-2, GPT-3, etc.reference: Chapter 3

Causal Mask
: A lower triangle mask that blocks future tokens from being seen in an autoregressive model.reference: Chapter 5

Context Length
: Maximum number of tokens the model can process at one time. GPT-2 is 1024, GPT-4 is 128K.reference: Chapter 7

Cross-Entropy Loss
: A loss function that measures the difference between the predicted probability distribution and the correct answer distribution in a classification problem. Standard loss in language modeling.reference: Chapter 7

Decoder
: The part of the transformer that generates sequences. GPT is a structure that uses only a decoder.reference: Chapter 6

Dropout
: A regularization technique that prevents overfitting by randomly setting some neu-

rons to 0 during training.reference: Chapter 4

Embedding
: The process of converting a discrete token ID into a continuous dense vector. Or the vector itself.reference: Chapter 4

Feed-Forward Network (FFN)
: Fully connected neural network for each position applied after attention in the transformer block. Usually 2 linear layers + activation function.reference: Chapter 6

GELU (Gaussian Error Linear Unit)
: A smooth version of ReLU. Smooth transition around zero using a Gaussian function. GPT model is used.reference: Chapter 6

Gradient
: Differential values for each parameter of the loss function. Indicates the direction of optimization.reference: Chapter 7

Greedy Decoding
: A generation strategy that selects only the tokens with the highest probability at each step. Fast but monotonous output.reference: Chapter 8

Head
: A unit that independently performs attention in multihead attention.reference: Chapter 5

Layer Normalization
: A technique for normalizing the mean and variance along the last dimension of a tensor. Essential for stabilizing transformer learning.reference: Chapter 4, Chapter 6

Learning Rate
: Hyperparameter that determines the step size of the gradient. If it is too large, it will diverge; if it is too small, it will converge slowly.reference: Chapter 7

Logits
: Raw output values before applying softmax. Usually the output of the last linear layer of the model.reference: Chapter 7

Loss Function
: A function that measures the difference between the model's predictions and the correct answer as a scalar. Optimization target for learning.reference: Chapter 7

Multi-Head Attention
: Learn various patterns simultaneously by running multiple attention heads in parallel.reference: Chapter 5

Parameter
: Weights that the model learns. W, b, etc. for linear layers.reference: Chapter 7

Perplexity
: Evaluation metrics for language models.$PPL = \exp(\text{loss})$. The lower the model,

the better.reference: Chapter 7

Positional Encoding
: How to inject ordering information into a transformer. sin/cos or learnable embeddings.reference: Chapter 4

Pre-LN / Post-LN
: Location of layer normalization. Pre-LN is applied before attention/FFN, and Post-LN is applied after. Pre-LN has excellent learning stability.reference: Chapter 6

Residual Connection
: A connection that adds an input to an output.$y = x + f(x)$. Essential for deep network training.reference: Chapter 6

Scaled Dot-Product Attention
: Attention in the form$Q \cdot K^T / \sqrt{d_k}$. The most basic operation of transformers.reference: Chapter 5

Self-Attention
: Attention between tokens in the same sequence. The core of Transformers.reference: Chapter 5

Softmax
: A function that converts a vector into a probability distribution.softmax$(x_i) = e^{x_i} / \sum_j e^{x_j}$.reference: Chapter 5

Temperature
: A parameter that controls the sharpness of the distribution when sampling. Lower is deterministic, higher is random.reference: Chapter 8

Token
: The smallest unit by which text is divided. Can be a word, subword, or byte.reference: Chapter 3

Tokenizer
: A module that converts text into token ID sequences and vice versa.reference: Chapter 3

Top-K Sampling
: A sampling strategy that considers only the K items with the highest probability among the following token candidates.reference: Chapter 8

Top-P (Nucleus) Sampling
: A sampling strategy that considers only tokens until the cumulative probability reaches P. Adaptive to distribution type.reference: Chapter 8

Transformer
: Vaswani et al. Attention-based neural network structure proposed by (2017). The foundation of the modern LLM.reference: Chapter 6

Unigram Tokenizer
: A tokenizer that assigns a probability to each token and learns with EM. Used by SentencePiece.reference: Chapter 3

Viterbi Algorithm
: Algorithm for finding optimal partitioning using dynamic programming. Used for decoding of Unigram tokenizer.reference: Chapter 3

Vocabulary
: The set of all tokens recognized by the tokenizer. Vocabulary size is usually 10,000 to 100,000.reference: Chapter 3

Weight Tying
: A technique to reduce the number of parameters by sharing the weights of the input embedding and output layer.reference: Chapter 7

Preface

Why I wrote this book

Since ChatGPT was released in late 2022, large language models (LLMs) have become the hottest topic in the field of artificial intelligence. Numerous developers and researchers quickly learned how to download, fine-tune, and integrate already-made models into services through HuggingFace Transformers. However, there are still few people who can confidently answer the question, "How does this model work internally?"

This book was written to answer that question. The goal of this book is to understand the framework of LLM by creating a tokenizer, implementing attention by hand, assembling transformer blocks, and running a learning loop. Instead of using an abstracted library, you can create it yourself from start to finish using only the basic operations of PyTorch. All the details encountered in the process are ultimately the path to "understanding" the LLM.

Features of this book

This book does not simply list theories.**Create the code first, After verifying with unit tests, The process is explained in a book.**.All Python code that appears in the book is actually executed, and only code that has passed pytest-based unit tests is included. Readers can download the entire code from the GitHub repository to run, modify, and learn directly.

Each chapter imports and uses the module implemented in the previous chapter. For example, the embedding module in Chapter 4 uses the tokenizer in Chapter 3, and the attention in Chapter 5 uses the embedding in Chapter 4. Through this "building structure", readers can experience step by step how the LLM is assembled. Mathematical formulas appear after intuitive explanations, and consideration is given to ensure that even beginners do not miss the flow.

Who should read it

The main readers of this book are developers who know basic Python syntax but are new to deep learning. This includes engineers who have used HuggingFace Transformers but do not know their inner workings, and researchers who have read the "Attention Is All You Need" paper but have never implemented it in code. Calculus and basic linear algebra are assumed, but advanced topics such as matrix differentiation are supplemented in the appendix.

summer 2026

dirmich

Acknowledgments

While writing this book, I received help from numerous open source projects and papers. Special thanks to Andrej Karpathy's nanoGPT, HuggingFace Tokenizers, and the excellent documentation from the PyTorch development team. These are pioneers who have dedicated themselves to the democratization of LLM.

We are also grateful to the numerous researchers on whom the code in this book is based. Ashish Vaswani and co-authors of the original Transformers paper are what started it all. Alec Radford and the OpenAI team demonstrated the potential of scaling with the GPT series. Tianyi Zhang and Jason Wei laid the foundation for instruction tuning and alignment.

I would also like to thank my family and friends. This book would not have been completed without the patience of those who waited silently while I wrote it.

Lastly, to all readers of this book. Your curiosity and passion create the future of LLM. Build it, experiment, fail, and try again. That is the essence of learning.

dirmich
Highmaru Press

Chapter 1

LLMWhat is

In this chapter, we look at what Large Language Models (LLMs) are, how they have evolved, and why it is worthwhile to try making your own. Because it focuses on concepts rather than code, even readers who are new to deep learning can read it without burden.

1.1 What is a language model?

Language model (language model)is a statistical model that assigns probabilities to sequences of words (or tokens). Simply put, it is a function$P(w_1, w_2, \ldots, w_n)$that determines "Is this sentence natural?" with probability. For example, "the cat is chasing the mouse" is natural, but "the mouse is chasing the cat" may be less natural depending on the context. The language model learns these probabilities.

Historically, language models started from the N-gram model, went through neural network-based models (RNN, LSTM), and developed dramatically with the emergence of Transformer in 2017. In particular, models that learned Internet-scale text with the simple goal of "next token prediction" showed amazing language understanding capabilities, ushering in today's LLM era.

Next token prediction

The learning goal of LLM is simple: predict the next token by looking at the tokens so far. If you write it as a formula

$$P(w\_t \mid w\_1, w\_2, \ldots, w\_t - 1)$$

Accurately estimating this conditional probability is the only goal of a language model. When this simple goal is combined with large amounts of data, complex capabilities such as question answering, code writing, and translation naturally emerge.
Why are these simple goals so powerful? To "understand" a language, you must have grammar, common sense, reasoning, and factual knowledge. To accurately predict the next token, the model must learn all of this implicitly. To predict "a planet" after "the Earth orbits the sun...", common sense in astronomy is needed, and to predict "2" after "1+1=", arithmetic is necessary.

1.2 intuition: How to think about language models

For readers who are new to language models, let me give an analogy. The language model is similar to "a person who has read a ton of books". This person was trained to guess the "next word" in every sentence. After training, the person "generates" an answer by stringing together the following words that seem most natural when asked a question.

Of course, it is different from a real person. Language models "match patterns" rather than "understand" them. But if you learn enough patterns, you will seemingly understand it. This is how models like GPT-4 provide surprising answers.

Does the language model "understand"?

This question is a matter of philosophical debate. From a functionalism perspective, it can be seen that "if you respond appropriately to input, you understand," or, conversely, it can be seen as, "if there is no internal representation, it is not understanding." This book focuses on "how it works" rather than philosophical debates. If you're interested in philosophy, look up Searle's "Chinese Room" thought experiment.

1.3 LLMDevelopment history of

1.3.1 Statistical Language Models (1990s – 2000s)

Early language models used a simple statistical technique called N-gram. It is a method that "estimates the probability of the next word by looking at the N-1 words immediately preceding it." For example, it learns through corpus statistics that "mat" is more likely to follow "the cat sat on the" than "banana".

```
# N-gram Key ideas of language models (simplified)
from collections import Counter

corpus = "the cat sat on the mat the cat ran".split()
# 2-gram (bigram) count
bigrams = [(corpus[i], corpus[i+1]) for i in range(len(corpus)-1)]
counts = Counter(bigrams)
# P("mat" | "the") estimate
p_mat_given_the = counts[("the", "mat")] / sum(1 for w1, w2 in bigrams if w1 == "the")
print(f"P(mat|the) = {p_mat_given_the:.2f}")
```

However, N-gram had serious limitations. There was a "sparsity" problem in that the context could only be seen briefly (N was usually 3~5), and unlearned combinations were assigned a probability of 0. A smoothing technique was developed to compensate for this, but it was not a fundamental solution.

Another problem was that it did not capture the "meaning of the words." Although "dog" and "puppy" have similar meanings, they are treated as completely different words in N-gram. To solve this problem, a neural network-based model that converts words into distributed representations has emerged.

1.3.2 Neural Network Language Models (2003 – 2017)

Bengio et al. (2003) first proposed a neural network-based language model. The method was to express words as low-dimensional dense vectors (embeddings) and put these embeddings into a neural network to predict the probability of the next word. Later, with the advent of RNN (Recurrent Neural Network) and LSTM (Long Short-Term Memory), it became possible to process longer contexts.

Word2Vec and Embedding

Mikolov et al. (2013)'s Word2Vec expressed words as 100∼300-dimensional vectors and showed that "words with similar meanings are close in the vector space." Famous example:$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$. This idea is the basis for the embedding we will implement in Chapter 4.

Limitations of RNN

RNN is a structure that transfers the hidden state of a previous point to the current point. In theory, it can handle infinite contexts, but in practice, it has the problem of vanishing or exploding gradients during backpropagation. As a result, the actual processable context length was only a few tens to hundreds of tokens. Additionally, only sequential processing was possible, making GPU parallelization inefficient. LSTM (Long Short-Term Memory) partially alleviated this problem with a gate mechanism, but it was not a fundamental solution. Contexts longer than 100 words were still difficult.

1.3.3 Rise of the Transformers (2017)

"Attention Is All You Need" published by Vaswani et al. in 2017 completely changed the landscape of language models. We proposed a Transformer structure that processes sequences only with an attention mechanism without RNN. There are two key innovations in Transformer.

1. **parallel processing**: Instead of processing sequentially like RNN, all positions in the sequence can be processed simultaneously, maximizing GPU utilization. This improved the learning speed by dozens of times.
2. **long distance dependency**: Attention computes direct connections between every pair of positions in a sequence, so it also captures relationships between distant tokens well. Even words that are 1,000 words apart can be referenced with a single attention.

1.3.4 GPT Series and Scaling Laws (2018) – today)

OpenAI's GPT series takes a pre-training approach with large-scale text using a transformer decoder structure. GPT-1 (2018) had 117 million parameters, but GPT-2 (2019) grew rapidly to 1.5 billion parameters, and GPT-3 (2020) grew rapidly to 175 billion parameters. In this process, it was discovered that performance improves predictably by increasing model size, data size, and computation volume.

Kaplan et al. (2020)'s scaling law showed that the loss follows a power law for model

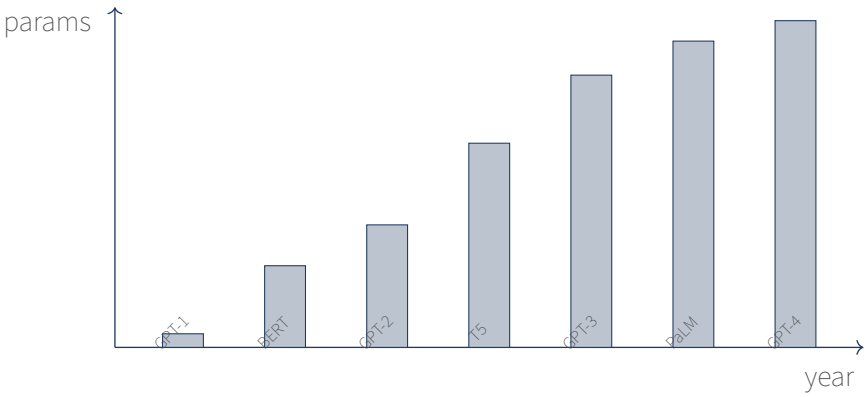


Figure 1.1: LLM Development trend of number of parameters (log scale, (Approximately). GPT-1 (2018) is 1.100 million, GPT-4 (2023) is estimated at 1.7 trillion.

size$N$, data size$D$, and computation amount$C$:

$$\mathcal{L}(N, D, C) \approx A \cdot N^{-\alpha\_N} + B \cdot D^{-\alpha\_D} + E \cdot C^{-\alpha\_C} + L\_\infty$$

The practical implications of this law are clear: to grow your model, you need to grow your data as well. If a 10 billion parameter model is trained with 1 billion tokens, it will overfit. Hoffmann et al. (2022)'s Chinchilla paper suggested that approximately 20 tokens per parameter$N$is ideal.

1.3.5 The Rise of Sort (2022) – today)

GPT-3 (2020) was powerful with 175 billion parameters, but did not provide helpful answers to user questions. It just predicted the "next token". The reason why ChatGPT was innovative in 2022 was not in the model size but in**array(alignment)**.

Reinforcement learning using human feedback (RLHF) transformed GPT-3 into a "useful chatbot." Afterwards, simpler and more efficient sorting techniques such as Direct Preference Optimization (DPO) and Constitutional AI emerged. Chapter 8 of Volume 2 implements these techniques directly.

1.4 LLMMain applications of

LLMs are used in a variety of fields:

1.4.1 Natural language processing traditional tasks

- **machine translation**: Google Translate, DeepL, etc. are based on Transformers.
- **summation**: Short summary of a long document. News, papers, meeting minutes, etc.
- **Sentiment analysis**: Determination of positive/negative text. Reviews, social media analysis.
- **Entity name recognition**: Extract people, places, organizations, etc. from text.

1.4.2 Create task

- **conversation**: ChatGPT, Claude, Gemini, etc.
- **Write code**: GitHub Copilot, Cursor. Various languages such as Python and JavaScript.
- **creation**: novel, poetry, screenplay. AI writer tools.
- **email/Write a document**: Business email, draft report.

1.4.3 bailiwick

- **medical treatment**: Diagnostic assistance, medical history summary, patient counseling.
- **law**: Case law search, contract review, legal advice.
- **education**: Personalized tutor, problem creation, and assignment grading.
- **finance**: Market analysis, report writing, fraud detection.

1.4.4 multimodal

Modern LLM processes not only text but also images, audio, and video. GPT-4V, Gemini, Claude 3, etc. can understand and explain images.

1.5 Why you should make it yourself

The question is legitimate: "Why create your own when you can just download the model from HuggingFace?" There are three answers.

first, depth of understandingis different. Calling`model.generate()`and knowing what happens inside it are two different levels of understanding. If you create it yourself, debugging, optimization, and customization are all possible. You can diagnose bottlenecks when inference speeds are slow, and know where to fix them when experimenting with new architectures.

second, controloccurs. Whether you want to increase inference speed, reduce memory, or experiment with a new architecture, the difference between the response ability of someone who knows the internals and someone who does not is heaven and earth. What if I use the HuggingFace model as is and get strange output? Anyone who knows the internals knows where to debug.

third, future proofc. The LLM field changes rapidly. New papers are released every week, and you need to have a solid foundation to understand them. Latest techniques such as RoPE, GQA, and SwiGLU are ultimately variations of attention, FFN, and regularization. If you build your basics through this book, you can naturally absorb advanced topics such as distributed learning, quantization, and sorting covered in Volume 2Make LLM-advanced.

Limitations of this book

This book does not aim to "create a real ChatGPT". The model created in Volume 1 is a small-scale GPT with 10 million~100 million parameters, and its quality is

about GPT-2 small. However, its structure is essentially the same as GPT-3 and GPT-4. “Size” is covered in Volume 2, and “Structure and Principle” is covered in Volume 1.
Why not make a larger model? Training a 175B model requires hundreds of GPUs over several weeks and costs hundreds of millions of dollars. This is difficult to follow in a book. Instead, if you learn the principle with a small model, you will learn that the same principle applies to the large model.

1.6 The model this book will create

After completing Volume 1, readers will be able to create the following models themselves.

- **architecture**: GPT style transformer decoder (6∼12 layers, 8 heads, 512∼768 embedding dimensions)
- **tokenizer**: Direct implementation of both BPE (Byte Pair Encoding) and Unigram
- **learning**: AdamW + cosine LR schedule + warm-up
- **generation**: Implements all Greedy, Top-K, Top-P, and Temperature sampling
- **evaluation**: Cross-entropy loss and perplexity calculation

Volume 2Make LLM-advancedbuilds on this foundation by adding distributed learning (DDP, FSDP, ZeRO), latest architecture (RoPE, GQA, SwiGLU, MoE), quantization (INT8/INT4/AWQ), alignment (SFT, RLHF, DPO, LoRA), and inference optimization (KV cache, PagedAttention).

1.7 Learning Roadmap

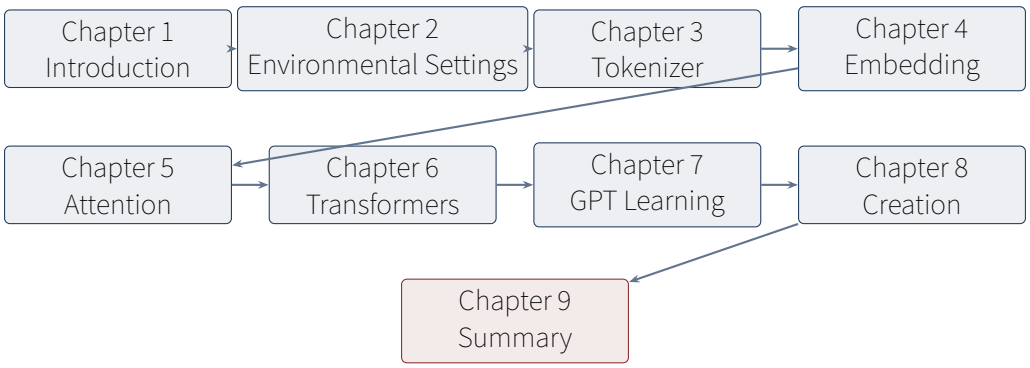


Figure 1.2: Volume 1 Learning Roadmap. Each chapter builds on the modules of the previous chapter. importuse it.

1.8 summation

- A language model is a model that assigns probabilities to text sequences, and its core goal is predicting the next token.
- From N-gram to RNN/LSTM, Transformer appeared in 2017, ushering in the LLM

era.

- Performance improves predictably when models, data, and operations grow according to scaling laws.
- After 2022, alignment (RLHF/DPO) has transformed "simple LM" into "useful chatbot".
- Creating it yourself gives you depth of understanding, control, and future-proofing.
- Volume 1 covers small-scale GPT, and Volume 2 covers ChatGPT-level techniques.

1.9 practice problems

1. Answer the following questions using ChatGPT, Claude, or Gemini. "If a language model is trained to predict the next token, how can it answer the question?" Evaluate the model's answer.
2. Consider how a human would rate the probability of the next word in the following sentence: "The sun is in the east…". "Soaring" will be more than 99%. Why is this probability so high?
3. Explain the "sparsity problem" among the limitations of the N-gram model. How does N-gram predict the next word of the untrained 3-gram "polar bear dancing"?
4. Explain why transformers are better than RNNs for parallel processing.
5. According to the scaling law, if the model is increased by 10 times, how much should the data be increased? (Based on Chinchilla)

In the next chapter, we set up Python, PyTorch, and the code repository for this book for full-scale implementation.

Chapter 2

Development environment settings

In this chapter, we set up the necessary development environment prior to full-scale LLM implementation. Explore Python, PyTorch, and the book's code repository structure, and run your first "Hello, LLM" program.

2.1 tools needed

equipment	use	Recommended version
Python	implementation language	3.10 or higher
PyTorch	Deep Learning Framework	2.0 or higher
NumPy	Numerical Computing	1.24 and higher
pytest	unit test	7.0+
git	version control	latest

GPU is optional. All code in this book is written to run on the CPU. However, for large-scale learning, CUDA-capable GPUs are strongly recommended. When discussing distributed learning in Volume 2, a multi-GPU environment is assumed.

2.2 PyTorch installation

PyTorch follows the installation guide on the official website. Choose the one that suits your environment between the CPU-only version and the CUDA version.

```
# CPU Only (all examples in this book are CPUoperation)
pip install torch --index-url https://download.pytorch.org/whl/cpu

# CUDA 12.1 Support (GPU when used)
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

To check installation, do the following:

```
import torch
print(torch.__version__)         # 2.x.x
print(torch.cuda.is_available()) # GPU Availability
```

2.3 repository structure

The code in this book has the following structure. The modules covered in each chapter are separated into separate files, so you can use only the parts you need.

```
make-llm-basic/
  src/makellm/
    __init__.py
    tokenizer/              # Chapter 3: tokenizer
      base.py
      bpe.py
      unigram.py
    model/                  # 4-Chapter 6: model structure
      embedding.py
      attention.py
      transformer_block.py
      gpt.py
    training/               # Chapter 7: learning
      dataset.py
      loss.py
      optimizers.py
      trainer.py
    inference/              # Chapter 8: inference
      sampler.py
      generate.py
    utils/                  # common utility
      config.py
      seed.py
      logging.py
  tests/                    # pytest unit testing
  book/                     # of this book LaTeX sauce
```

2.4 first example: Dealing with tensors

Before full-scale implementation, let's review the basics of PyTorch tensors. Tensors operate almost identically to NumPy arrays, except that they support GPU operations and automatic differentiation.

```
import torch

# Tensor creation
x = torch.randn(2, 3)         # Sampling from a standard normal distribution
print(x.shape)                # torch.Size([2, 3])
print(x.dtype)                # torch.float32

# calculation
y = x @ x.T                   # matrix multiplication: [2, 3] @ [3, 2] -> [2, 2]
z = x.sum()                   # Sum of all elements (scalar)

# automatic differentiation
w = torch.randn(3, requires_grad=True)
loss = (w * w).sum()          # L2 norm^2
loss.backward()               # backward
print(w.grad)                 # d(loss)/d(w) = 2*w
```

Autograd

One of PyTorch's most powerful features. All operations performed on tensors are internally recorded as a "computation graph", and gradients for all parameters are automatically calculated`loss.backward()`once. There is no need to derive the differential equation ourselves. All models in this book rely on autograd.

2.5 Ensure reproducibility

Deep learning experiments have a lot of randomness, so reproducibility is important. If you fix the seed, you can run the same code multiple times and get the same results.

```
from makellm.utils import set_seed
set_seed(42)  # Python, NumPy, PyTorch Set up your seeds in one step
```

This function calls Python's`random`, NumPy's`numpy.random`, and PyTorch's`torch.manual_seed`. If CUDA exists, the CUDA seed is also set.

2.6 Run unit tests

The code for each chapter is provided with pytest-based unit tests. After modifying the code, be sure to run tests to make sure there are no regressions.

```
cd make-llm-basic
pytest tests/ -v
```

```
======================== 48 passed in 2.47s ========================
```

If all 48 tests pass, the environment setup is complete.

2.7 summation

- Installed PyTorch 2.x and pytest.
- I understand the repository structure: Code is separated by module in`src/makellm/`.
- We reviewed the basics of tensor operations and autograd.
- Use`set_seed()`for reproducibility.
- You can run unit tests with `pytest tests/`.

In the next chapter, we will implement the tokenizer in earnest. This first conversion of text to numbers is the starting point for all LLMs.

Chapter 3

Creating a tokenizer

In this chapter, we directly implement**Tokenizer (tokenizer)**, which converts text to a sequence of numbers. It covers two major algorithms, BPE (Byte Pair Encoding) and Unigram, and examines each learning, encoding, and decoding process through code.

3.1 Why do you need a tokenizer?

Neural networks only understand numbers. The string "Hello, world!" cannot be directly entered into the model. The first step is to convert the text into a sequence of integers, which is handled by**tokenizer**. The simplest method is a "per-character tokenizer," which treats every character as a separate token. However, this makes the sequence too long and it is difficult to get the meaning of the words.

The opposite extreme is a "word-level tokenizer," which treats words separated by spaces as a single token. However, it has an exploding vocabulary size (hundreds of thousands of words in English alone) and cannot handle untrained words (out-of-vocabulary, OOV).

Modern LLMs use the**Subword (subword)**tokenizer, which is somewhere between these two extremes. Words that appear frequently are divided into one token, and rare words are divided into smaller subwords. "unhappiness" can be broken down into "un" + "happiness" or "un" + "happi" + "ness". This method solves the OOV problem while maintaining an appropriate vocabulary size.

3.1.1 Comparison of three approaches

method	vocabulary size	OOV treatment	sequence length
Character unit	small (~100)	none	very long
Word units	very large (~100K)	not processed	short
Subword	Middle (~10K~50K)	Decomposition into partial words	Middle

3.2 special token

Every tokenizer has some**Special token (special token)**. Our implementation uses four:

token	ID	use
<pad>	0	Padding to match sequence length within batch
<bos>	1	Indicates the beginning of sequence (Beginning Of Sequence)
<eos>	2	Marks the end of the sequence (End Of Sequence)
<unk>	3	Token not in vocabulary (Unknown)

These tokens have fixed indices, so they can be treated specially by the model. For example, if<eos>is encountered during creation, inference stops, and the<pad>position is ignored in the loss calculation.

Reason for fixing special token ID

Fixing the ID of the special token to 0~3 simplifies the model code. Hard coding such as "`if token_id == 0: mask_out`" is possible. Additionally, checkpoint compatibility is guaranteed: even if the tokenizer is retrained, the special token ID is always the same, so model weights can be reused as is.

3.3 base class

The interface that all tokenizers must follow is defined as an abstract class.

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class SpecialTokens:
    pad: str = "<pad>"   # ID 0
    bos: str = "<bos>"   # ID 1
    eos: str = "<eos>"   # ID 2
    unk: str = "<unk>"   # ID 3

    @property
    def tokens(self) -> list[str]:
        return [self.pad, self.bos, self.eos, self.unk]

    @property
    def pad_id(self) -> int: return 0
    @property
    def bos_id(self) -> int: return 1
    @property
    def eos_id(self) -> int: return 2
    @property
    def unk_id(self) -> int: return 3


class Tokenizer(ABC):
    def __init__(self, special_tokens: SpecialTokens | None = None):
        self.special = special_tokens or SpecialTokens()
        self.id_to_token: list[str] = list(self.special.tokens)
        self.token_to_id: dict[str, int] = {t: i for i, t in enumerate(self.id_to_token)}

    @abstractmethod
```

```
    def _train(self, corpus: list[str], vocab_size: int) -> None:
        """Vocabulary learning (implemented in child class)"""

    @abstractmethod
    def _tokenize(self, text: str) -> list[str]:
        """Split text into list of tokens (implemented in child class)"""

    def encode(self, text: str, add_bos=False, add_eos=False) -> list[int]:
        tokens = self._tokenize(text)
        ids = [self.token_to_id.get(t, self.special.unk_id) for t in tokens]
        if add_bos: ids = [self.special.bos_id] + ids
        if add_eos: ids = ids + [self.special.eos_id]
        return ids

    def decode(self, ids: list[int]) -> str:
        tokens = [self.id_to_token[i] for i in ids
                  if self.id_to_token[i] not in self.special.tokens]
        raw = self._tokens_to_bytes(tokens)
        return self._postprocess(raw.decode("utf-8", errors="replace"))
```

Role of abstract class

`Tokenizer` defines an "interface". Child classes (BPE, Unigram) must implement `_train` and `_tokenize`. In this way, even if the learning algorithm is different, the `encode/decode` API remains the same, so the upper code (dataset, trainer) operates regardless of the type of tokenizer.
This is the value of "polymorphism". The trainer only needs to call `tokenizer.encode(text)`, and it doesn't care whether BPE or Unigram is running internally. If you later add new algorithms such as "SentencePiece" or "WordPiece", the trainer code does not need to be modified.

3.4 BPE tokenizer

BPE(Byte Pair Encoding) is an algorithm proposed for data compression in 1994 that was introduced to neural network machine translation by Sennrich et al. in 2016. Used by GPT-2, GPT-3, LLaMA, etc.

3.4.1 Algorithm Intuition

The core idea of BPE is simple:

1. Decomposes text into bytes (any character can be represented, no OOV).
2. Finds the most frequently occurring pair of adjacent byte pairs.
3. Merge the pair into a new token.
4. Repeat 2~3 until the target vocabulary size is reached.

Initially, all text is a "sequence of bytes". For example, "lower" is expressed as `[l, o, w, e, r]`. As you progress:

- Merge if "lo" appears frequently: `[lo, w, e, r]`

- Merge if “er” appears frequently:`[lo, w, er]`
- Merge if “low” appears frequently:`[low, er]`
- Merge if “lower” appears frequently:`[lower]`

Ultimately, “lower” becomes a token, but the rare word remains a subword. “lowering” can be “lower” + “ing”.

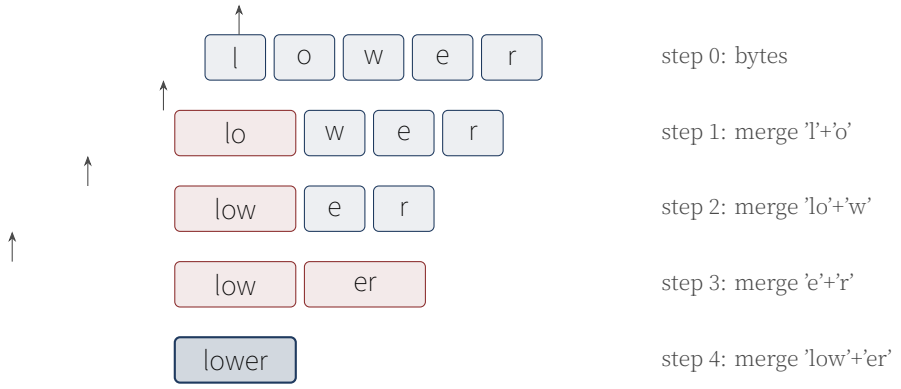


Figure 3.1: BPE learning process. “lower”The process of starting from bytes and gradually merging them into one token..

3.4.2 Why Byte Unit??

Starting with GPT-2, BPE operates on a “byte-by-byte” basis. Previously we used character units, but there was a problem:

- Characters that are not in the vocabulary, such as emojis and rare Chinese characters, are processed as<unk>.
- Character sets become too large in multilingual corpora.

In byte units, all UTF-8 characters can be expressed in 1~4 bytes, and the “byte” part of the vocabulary is always fixed at 256. OOV virtually disappears.

3.4.3 avatar: learning

```python
from collections import Counter
from .base import Tokenizer

class BPETokenizer(Tokenizer):
    def __init__(self, special_tokens=None):
        super().__init__(special_tokens=special_tokens)
        self.merges: list[tuple[str, str]] = []
        self._merge_rank: dict[tuple[str, str], int] = {}

    def _train(self, corpus: list[str], vocab_size: int) -> None:
        # 1. special token + Construct an initial vocabulary of 256 bytes
        self.id_to_token = list(self.special.tokens)
        for b in range(256):
            self.id_to_token.append(chr(b))
        self.token_to_id = {t: i for i, t in enumerate(self.id_to_token)}

        # 2. Split the corpus into words
```

```
        word_freqs = Counter()
        for text in corpus:
            for word in self._pre_tokenize(text):
                word_freqs[word] += 1

        # 3. Convert each word to a list of byte tokens
        word_to_tokens = {
            w: [chr(b) for b in w.encode("utf-8")]
            for w in word_freqs
        }

        # 4. merge loop
        self.merges = []
        target = vocab_size - len(self.id_to_token)
        for _ in range(max(target, 0)):
            pair_freqs = Counter()
            for word, freq in word_freqs.items():
                toks = word_to_tokens[word]
                for i in range(len(toks) - 1):
                    pair_freqs[(toks[i], toks[i+1])] += freq
            if not pair_freqs:
                break
            best = max(pair_freqs.items(), key=lambda x: (x[1], x[0]))
            if best[1] < 2:
                break
            new_token = best[0][0] + best[0][1]
            self.merges.append(best[0])
            self.id_to_token.append(new_token)
            self.token_to_id[new_token] = len(self.id_to_token) - 1
            for word in word_to_tokens:
                word_to_tokens[word] = self._merge_in_list(
                    word_to_tokens[word], best[0], new_token
                )

        self._merge_rank = {p: i for i, p in enumerate(self.merges)}
```

Merge priority stability

The reason for sorting from `max(pair_freqs.items(), key=lambda x: (x[1], x[0]))` to `(x[1], x[0])` tuples is because of tie processing. When there are multiple pairs with the same frequency, the oldest pair in alphabetical order is selected to make the result deterministic. This way, if you learn with the same corpus, you will always get the same merging order.

3.4.4 encoding

To apply learned merging rules to new text, apply the merge with the highest priority (learned first) first.

```
    def _tokenize(self, text: str) -> list[str]:
        tokens = []
        for word in self._pre_tokenize(text):
            bs = word.encode("utf-8")
            word_tokens = [chr(b) for b in bs]
```

```
            word_tokens = self._apply_merges(word_tokens)
            tokens.extend(word_tokens)
        return tokens

    def _apply_merges(self, tokens: list[str]) -> list[str]:
        """Apply learned merge rules in priority order."""
        while len(tokens) >= 2:
            # most rankFind low (high priority) pairs
            best_idx, best_rank = -1, len(self.merges) + 1
            for i in range(len(tokens) - 1):
                rank = self._merge_rank.get((tokens[i], tokens[i+1]), -1)
                if 0 <= rank < best_rank:
                    best_rank, best_idx = rank, i
            if best_idx < 0:
                break
            merged = tokens[best_idx] + tokens[best_idx + 1]
            tokens = tokens[:best_idx] + [merged] + tokens[best_idx + 2:]
        return tokens
```

3.4.5 Preprocessing: White space processing

BPE requires special handling of whitespace to preserve word boundaries. GPT-2 replaces spaces with the special character Ġ(U+0120). We replace it with U+2581( ) following the SentencePiece method.

```
    def _pre_tokenize(self, text: str) -> list[str]:
        text = text.replace(" ", "□")
        text = text.replace("Ġn", "□").replace("Ġt", "□")
        while "□□" in text:
            text = text.replace("□□", "□")
        if not text.startswith("□"):
            text = "□" + text
        parts = text.split("□")
        return ["□" + p for i, p in enumerate(parts) if i > 0 and p]
```

Trap of space substitution

If you replace “Hello world” with “Hello world” and then split, it becomes`["□Hello", "□world"]`. This is attached to the front of each word, so when decoding, if you replace it with a space, the original sentence is restored.
However, you must be careful if there are punctuation marks, such as “Hello, world!”. A simple split creates “Hello,”, making the comma part of the word. In reality, GPT-2 separates words/punctuation marks using regular expression patterns, but this is omitted in this book for simplicity.

3.4.6 decoding: byte restoration

The tricky part of BPE is decoding. Since the token is a sequence of`chr(byte)`, the code point of each character must be treated as a byte, the original byte stream must be restored, and then decoded into UTF-8.

```
    def _tokens_to_bytes(self, tokens: list[str]) -> bytes:
```

```
        """BPE Convert tokens to bytes. Code point of each character as byte."""
        out = bytearray()
        for tok in tokens:
            for c in tok:
                out.append(ord(c) & 0xFF)
        return bytes(out)
```

For example, if there is a " Hello" token, (U+2581) is`0xE2 0x96 0x81`in UTF-8, so it is 3 bytes, but during BPE learning, the " " character itself was processed as 3 individual byte tokens, so it is expressed as`chr(0xE2) + chr(0x96) + chr(0x81)`. When decoding, if you extract the bytes from each`chr(b)`to`ord()`, you get the original byte stream`0xE2 0x96 0x81 0x48 0x65 0x6C 0x6C 0x6F`, and if you decode this to UTF-8, you get "Hello".

3.5 Unigram tokenizer

UnigramTokenizer is a method proposed by Kudo (2018) as part of SentencePiece. If BPE is "frequency-based merging," Unigram is "probability-based partitioning."

3.5.1 intuition

Unigram assigns a probability to each token. There are several possible divisions for the sentence "lower":

- `[l, o, w, e, r]`: 5 tokens
- `[lo, w, er]`: 3 tokens
- `[low, er]`: 2 tokens
- `[lower]`: 1 token

The probability of each split is calculated as the product of the token probabilities. Select the split with the highest probability. If "lower" is in the vocabulary and with high probability,`[lower]`is chosen, but if not, it is split into smaller units.

3.5.2 algorithm

1. Extract all substrings from the corpus as candidate tokens and count their frequencies.
2. Converts frequency to probability and assigns a log probability score to each token.
3. EM(Expectation-Maximization) iteration:
    - E-step: Search for optimal segmentation of each word with Viterbi algorithm.
    - M-step: Update token probability with split result.
4. Reduce vocabulary by removing tokens with the lowest scores.

3.5.3 Viterbi decoding

Dynamic programming finds the "token split with the highest total log probability" for a given word.

```
    def _viterbi(self, word: str) -> tuple[list[str], float]:
```

```
        n = len(word)
        # dp[i] = (maximum score, previous index, old token)
        dp = [(-float("inf"), -1, "") for _ in range(n + 1)]
        dp[0] = (0.0, -1, "")
        for i in range(1, n + 1):
            for j in range(max(0, i - 16), i):
                substr = word[j:i]
                score = self.token_scores.get(substr, -20.0)
                cand = dp[j][0] + score
                if cand > dp[i][0]:
                    dp[i] = (cand, j, substr)
        # backtracking
        path, i = [], n
        while i > 0:
            _, j, tok = dp[i]
            path.append(tok)
            i = j
        path.reverse()
        return path, dp[n][0]
```

Viterbi's complexity

The secret to shortening it to $O(n \cdot L)$ instead of $O(n^2)$ is the `max(0, i-16)` part. Since the token length rarely exceeds 16, the search space is reduced by limiting the range of j. The actual SentencePiece also uses similar optimization.

3.6 BPE vs Unigram comparison

	BPE	Unigram
Learning method	frequency-based merging	probability-based segmentation (EM)
Encoding	applying merge rule order	optimal splitting with Viterbi
Deterministic	O (same input, same output)	O
Supports multi-splitting	X	O (sampling possible)
Main uses	GPT-2, GPT-3, LLaMA	T5, ALBERT, mBART

3.7 Tokenizer Test

Once implementation is complete, verify it with unit tests. The most important test is that**Round Trip (round-trip) test**: the original text must be restored when encoded and then decoded.

```
def test_encode_decode_roundtrip():
    tok = BPETokenizer()
    tok.train(CORPUS, vocab_size=200)
    text = "the quick fox"
    ids = tok.encode(text)
    decoded = tok.decode(ids)
    assert " ".join(decoded.split()) == "the quick fox"
```

```python

def test_bos_eos_addition():
    tok = BPETokenizer()
    tok.train(CORPUS, vocab_size=100)
    ids = tok.encode("hello", add_bos=True, add_eos=True)
    assert ids[0] == tok.special.bos_id
    assert ids[-1] == tok.special.eos_id

def test_unknown_token_handled():
    """Even characters that are not in the vocabulary are processed byte by byte. unkshould not occur."""
    tok = BPETokenizer()
    tok.train(CORPUS, vocab_size=100)
    ids = tok.encode("hi")
    assert tok.special.unk_id not in ids  # Because it is decomposed into bytes unk doesn't exist

def test_save_load(tmp_path):
    tok = BPETokenizer()
    tok.train(CORPUS, vocab_size=150)
    path = tmp_path / "tok.json"
    tok.save(path)
    tok2 = BPETokenizer.load(path)
    text = "the quick fox"
    assert tok.encode(text) == tok2.encode(text)
```

Testing Philosophy

All modules in this book have three levels of testing:

- **Geometry test (shape test)**: Whether the shape of the tensor matches expected.
- **Numerical test (numeric test)**: Compare with hand calculation results for small inputs.
- **integration testing**: Operates end-to-end when connecting multiple modules.

Only code that passes these three tests is included in the book.
Round-trip testing is an example of integration testing. Although it seems self-evident that "encoding and then decoding will yield the original text", in reality, bugs can occur at various stages, such as handling whitespace, removing multibyte characters, and special tokens. If this test passes, you can be confident that the entire pipeline is operating correctly.

3.8 practice: Training tokenizer with real corpora

If you run the book's`scripts/tiny_train.py`demo, you can actually see the process of learning the BPE tokenizer.

```
cd make-llm-basic
python scripts/tiny_train.py --vocab 500
```

Example output:

```
[1/5] Learning tokenizer...
    complete: 308 tokens, 1.2 seconds
```

```
    sample: 'the quick brown fox jumps over the lazy dog'
    → 18 tokens: [264, 282, 291, 267, 261, 110, ...]
```

3.9 summation

- The tokenizer is the first step in LLM that converts text into a sequence of integers.
- The subword method strikes a balance between vocabulary size and OOV.
- BPE is frequency-based merging, Unigram is probability-based splitting.
- Special token<pad>/<bos>/<eos>/<unk>is fixed to ID 0∼3.
- Byte-wise BPE virtually eliminates OOV.
- Find the optimal division of Unigram in$O(n)$using the Viterbi algorithm.
- All implementations are verified with pytest unit tests.

3.10 practice problems

1. What happens if we merge the “least frequent pair” instead of the “most frequent pair” when learning BPE? Predict the results and experiment.
2. What happens if you change the`score = -20.0`penalty value to`-5.0`in the Unigram tokenizer?
3. Let’s say you are learning the Korean corpus “Hello, nice to meet you” using BPE. What is the difference between a vocabulary size of 100 and 1000?
4. What tokens are lost if Viterbi reduces the token length cap to 4 instead of 16?
5. <bos>Think about why you need a token. Will the model work without it?

In the next chapter, we implement an embedding layer that converts token IDs into dense vectors.

Chapter 4

Embedding and Positional Encoding

In this chapter, we implement**Embedding (embedding)**, which converts token IDs into dense vectors, and**Positional encoding (positional encoding)**, which injects sequence order information into the model.

4.1 Why do you need embedding?

In Chapter 3, the tokenizer changed "Hello" to the integer ID 247. However, if you put this integer directly into a neural network, a problem arises. First, there is no guarantee that the integers 247 and 248 are "similar words." "cat" might be 247, "dog" might be 248, and "zebra" might be 248. Second, operations that multiply or add integers have no meaning. "cat + 1 = dog" does not hold.

The solution is to convert each token ID to**dense vector**. When the vocabulary size is$V$and the embedding dimension is$d$, the embedding is a learnable matrix of size$V \times dE$. When token ID$i$comes in, the$i$th row of$E$is retrieved. This row vector becomes the "semantic representation" of the token.

Why "dense"?

One-hot encoding represents tokens as sparse vectors of vocabulary size$V$. For example, for a vocabulary of 10,000, ID 247 is a vector consisting of 246 0s, 1 1, and the remaining 0s. Multiplying this by$E$is the same as getting the 247th row of$E$. Embedding is an efficient implementation that performs this "one-hot×matrix" with a single lookup operation.
As learning progresses, words with similar meanings become closer in the vector space. "cat" and "dog" are close, and "cat" and "car" are far. This is the magic of embeddings: before training, they are random vectors, but after training, they become meaningful coordinates.

4.2 Mathematical definition of embedding

Given an embedding matrix$E \in \mathbb{R}^{V \times d}$, the embedding of token ID$i$is$E[i] \in \mathbb{R}^d$. This is simply an operation that selects the$i$th row of$E$, and is differentiable (when backwards, only the selected row receives the gradient).

For batch input$\mathbf{x} \in \mathbb{Z}^{B \times L}$(batch$B$, sequence length$L$), the embedding output will be$\mathbf{X} \in \mathbb{R}^{B \times L \times d}$.

4.3 Token Embedding avatar

PyTorch's`nn.Embedding`performs exactly this lookup operation. We wrap this thinly and add initialization logic.

```python
import math
import torch.nn as nn

class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size: int, d_model: int, pad_id: int = 0):
        super().__init__()
        self.vocab_size = vocab_size
        self.d_model = d_model
        self.pad_id = pad_id
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_id)
        self._init_weights()

    def _init_weights(self):
        # normal distribution N(0, 1/sqrt(d_model)) reset (GPT-2 styles)
        nn.init.normal_(self.embedding.weight, mean=0.0,
                        std=1.0 / math.sqrt(self.d_model))
        # Padding token to 0
        with torch.no_grad():
            self.embedding.weight[self.pad_id].fill_(0.0)

    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
        # [batch, seq] -> [batch, seq, d_model]
        return self.embedding(token_ids)
```

Importance of initialization

In deep learning, weight initialization determines the success or failure of learning. If it is too large, the activation value explodes, and if it is too small, the gradient disappears. GPT-2 uses a normal distribution with standard deviation$1/\sqrt{d_model}$, which is a reasonable choice so that the norm of the initial embedding vector is approximately 1.
Why$1/\sqrt{d}$? When each element of ad-dimensional vector is$\mathcal{N}(0, \sigma^2)$, the expected value of the vector norm is$\sqrt{d} \cdot \sigma$. If$\sigma = 1/\sqrt{d}$, norm is near 1. This way, the scale of the embedding vector is similar to the layer output, making learning stable.

4.3.1 padding_idxrole of

The`padding_idx`parameter of`nn.Embedding`always keeps the embedding of the padding token at 0. If you do this:

- The output of the padding position becomes 0 and does not affect attention calculation.
- When going backward, the gradient of the padding row becomes 0 and the weight is not updated.

4.4 Positional Encoding

Transformer processes sequences using only attention, but attention itself has no concept of "order". Even if you mix the inputs, the output (rearranged) is the same. This is called**Order invariance (permutation invariance)**. In natural language, order determines meaning ("dog bites man" vs "man bites dog"), so order information must be explicitly injected.

4.4.1 intuition: Why Order Matters?

Attention learns "how much attention each token will pay to other tokens." However, there is no distinction between the "first token" and the "last token". In the sentence "I ate rice", "I ate" must be connected to the subject "I", but it is difficult for attention to learn this connection without the information that "the 5th token sees the 1st token".

Positional encoding adds a unique "position vector" to each position, allowing the model to distinguish between positions.

4.4.2 Sinusoidal Position encoding (Vaswani et al. 2017)

Method proposed in the original Transformer paper. Encode the position with sine and cosine functions.

Sinusoidal Positional Encoding

$$PE\_(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d\_model}}\right)$$

$$PE\_(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d\_model}}\right)$$

Where$pos$is the location and$i$is the dimension index. Even dimensions are sin, odd dimensions are cos.

The advantage of this method is that there are no learnable parameters. Additionally, a linear relationship for relative positions is established, making it easy for the model to learn. Since$\sin(\alpha+\beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$, the encoding at position$pos + k$is expressed as a linear transformation of the encoding at positionpos.

```python
class PositionalEmbedding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 512, mode: str = "learned"):
        super().__init__()
        self.mode = mode
        if mode == "sinusoidal":
            pe = torch.zeros(max_len, d_model)
            position = torch.arange(0, max_len).unsqueeze(1).float()
            div_term = torch.exp(
                torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
            )
            pe[:, 0::2] = torch.sin(position * div_term)
            pe[:, 1::2] = torch.cos(position * div_term)
            self.register_buffer("pe", pe.unsqueeze(0))  # [1, max_len, d_model]
        else:  # learned
            self.pe = nn.Embedding(max_len, d_model)
            nn.init.normal_(self.pe.weight, mean=0.0, std=1.0/math.sqrt(d_model))
```

```
    def forward(self, seq_len: int) -> torch.Tensor:
        # [1, seq_len, d_model] return
        if seq_len > self.max_len:
            raise ValueError(f"Sequence length {seq_len} exceeds max_len {self.max_len}")
        if self.mode == "sinusoidal":
            return self.pe[:, :seq_len, :]
        else:
            positions = torch.arange(seq_len, device=self.pe.weight.device)
            return self.pe(positions).unsqueeze(0)
```

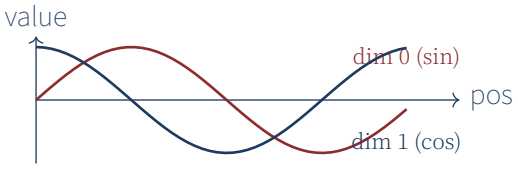


Figure 4.1: Sinusoidal First two dimensions of position encoding. sinclass cosIt changes periodically depending on location.

4.4.3 Learnable location embeddings (GPT-2 styles)

GPT-2, GPT-3, etc. use learnable embedding vectors for each location. This method is more flexible when there is sufficient data, but it is difficult to respond to lengths that have not been seen during training (e.g., 1024 during training, but 2048 during inference).

Sinusoidal vs Learned

- **Sinusoidal**: No parameters, extrapolation possible (corresponds to sequences longer than the length seen during learning). However, actual performance tends to be slightly lower than that of learnable methods.
- **Learned**: Slightly better performance, but fixed length. Extrapolation difficult.

Newer models combine the advantages of both methods using Rotary Position Embedding (RoPE). This is covered in Volume 2, Chapter 5.

4.5 Embedding composition

Token embeddings and position embeddings are added to create the final input representation. Vaswani et al. suggested scaling the embedding by multiplying it by $\sqrt{d\_model}$ to keep the scale similar to the positional embedding.

```
class EmbeddingStack(nn.Module):
    def __init__(self, vocab_size, d_model, max_len=512,
                 pad_id=0, pos_mode="learned", dropout=0.1):
        super().__init__()
        self.token_emb = TokenEmbedding(vocab_size, d_model, pad_id=pad_id)
        self.pos_emb = PositionalEmbedding(d_model, max_len, mode=pos_mode)
        self.dropout = nn.Dropout(dropout)
        self.scale = math.sqrt(d_model)

    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
```

```
        batch, seq = token_ids.shape
        tok = self.token_emb(token_ids) * self.scale  # scaling
        pos = self.pos_emb(seq)
        return self.dropout(tok + pos)
```

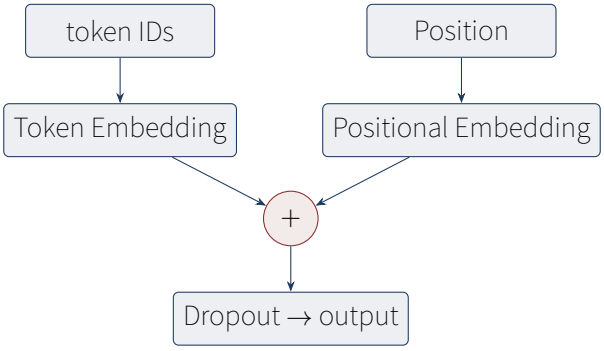


Figure 4.2: EmbeddingStack structure. Add token embedding and location embedding and then apply dropout..

4.5.1 Why do you need scaling??

The token embedding is initialized to$\mathcal{N}(0, 1/d)$, so the variance of the elements is$1/d$. The elements of the positional embedding (sinusoidal) are in the range$[-1, 1]$and have a variance of approximately$1/2$. When added without scaling, the location information overwhelms the token information.

Multiplying by$\sqrt{d}$makes the variance of the token embeddings1, which is balanced with the positional embeddings.

4.6 Layer Normalization introduction

Layer normalization (Layer Normalization)is used to stabilize learning in Transformer. It will also appear in Attention in the next chapter, so it is introduced here.

Layer Normalization

$$\mathrm{LN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Where$\mu, \sigma^2$is the mean and variance calculated along the last dimension.$\gamma, \beta$is a learnable parameter.

In PyTorch,`nn.LayerNorm(d_model)`is used as a single line. Volume 2 also covers RMSNorm, a variant of LayerNorm.

LayerNorm vs BatchNorm

BatchNorm normalizes along the batch dimension, so there is a problem with different statistics during learning and inference. LayerNorm operates regardless of batch size by normalizing within each sample. This is why LayerNorm is used in LLM: it works stably even in inference with a batch size of 1.

4.7 test

The core tests of the embedding module are checking the output shape and gradient flow.

```python
def test_token_embedding_shape():
    emb = TokenEmbedding(vocab_size=100, d_model=32)
    ids = torch.randint(0, 100, (2, 8))  # [batch=2, seq=8]
    out = emb(ids)
    assert out.shape == (2, 8, 32)

def test_positional_sinusoidal():
    pe = PositionalEmbedding(d_model=32, max_len=64, mode="sinusoidal")
    out = pe(seq_len=16)
    assert out.shape == (1, 16, 32)

def test_embedding_stack():
    stack = EmbeddingStack(vocab_size=100, d_model=32, max_len=64, dropout=0.0)
    ids = torch.randint(0, 100, (2, 8))
    out = stack(ids)
    assert out.shape == (2, 8, 32)

def test_positional_exceeds_max_len_raises():
    pe = PositionalEmbedding(d_model=32, max_len=32, mode="learned")
    with pytest.raises(ValueError):
        pe(seq_len=64)  # max_len over
```

4.8 practice: Embedding visualization

If you project the embeddings learned from`scripts/tiny_train.py`into 2D using PCA, you can see how the words are distributed. After sufficient learning, you can see that similar words are grouped close together.

4.9 summation

- Embedding is a learnable lookup table that converts token IDs into dense vectors.
- initialization uses$N(0, 1/\sqrt{d_model})$, and the padding token is set to 0.
- Positional encoding injects ordering information into the transformer. There are two methods: sinusoidal and learned.
- Add token embeddings and position embeddings and apply dropout to create the final input representation.
- $\sqrt{d_model}$Balance the variance of the two embeddings by scaling.
- LayerNorm is a batch size-independent regularization that is essential for stabilizing LLM learning.

4.10 practice problems

1. What happens if the embedding dimension d is too small (e.g. 8)? What happens if it's too big (e.g. 4096)?
2. What happens if you change the constant `10000` to 100 in sinusoidal positional encoding?
3. How would learning change if we did not multiply the token embeddings by $\sqrt{d\_model}$?
4. What happens if `max_len` of the learnable position embedding is set to 1024, learned, and then input a 2048 length sequence during inference?
5. How does the output change if you change the ϵ value of LayerNorm from 10^{-5} to 10^{-1}?

In the next chapter, we implement an attention mechanism that takes this embedding as input and learns "how much attention each token will pay to other tokens."

Chapter 5

attention mechanism

In this chapter, we implement the**Attention (attention)**mechanism, the core of Transformer. Starting from Scaled Dot-Product Attention, we look at it step by step to Multi-Head Attention.

5.1 intuition: What is Attention?

What does “it” refer to in the sentence “The cat sat on the mat because**it**was tired”? A person easily knows that it is a “cat”. How does a computer know? Attention is the mechanism that solves this problem. The idea is to learn how much attention each word pays to every other word, so that “it” gives a higher weight to “cat”.

As a formula, attention is an operation that calculates the similarity with all keys$K$for a given query$Q$and then adds the weighted value$V$based on the similarity.

General definition of attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d\_k}}\right) \cdot V$$

- Q: Query (representation of the token currently being viewed)
- K: key (“label” of other tokens)
- V: value (the “content” of other tokens)
- d_k: Dimension of the key. The reason for dividing by$\sqrt{d_k}$is to prevent softmax from becoming saturated due to the large dot product.

5.1.1 intuitive analogy: library search

Imagine a situation where you are looking for a book in a library. Go to the library with the information you want (query$Q$). Each book has a title and keywords (key$K$), and content (value$V$). The library system compares your query to the key of each book, calculates the similarity, and provides an answer by weighted sum of the contents of the most similar books.

Attention is what makes this process differentiable. “Similarity” is performed as a dot product, and “weighted summation” is performed as a softmax weight.

5.2 Q, K, VWhere does it come from?

In transformers,$Q, K, V$are all created as linear transformations from the same input$x$.

$$Q = xW\_Q, \quad K = xW\_K, \quad V = xW\_V$$

Here,$W_Q, W_K, W_V$is a learnable matrix. This way, you can learn three perspectives on the same input: “Question”, “Label”, and “Content”.

Why three different matrices?

The reason for converting toW_Q, W_K, W_Vinstead of using the same$x$three times is because of “role division”. The query should express “what to look for”, the key should express “what is there”, and the value should express “the actual content”. It is difficult to play all three roles with the same vector. Through linear transformation, it is projected into a “representation space” suitable for each role.

5.3 scaling: why – Divide by?

$Q \cdot K^T$is the dot product of$d_k$dimension vectors. When$d_k$is large, the inner product value becomes large. If the elements ofQ, Khave mean 0 and variance 1, the variance of the dot product$\sum_i = 1^{d_k} Q_i K_i$isd_k.

When the dot product is large, the softmax output approaches one-hot (one value is 1, the other is 0). When this happens, the gradient becomes almost 0 and learning stops. When divided by$\sqrt{d_k}$, the variance becomes 1 and softmax is distributed smoothly.

Effect of scaling

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d\_k}}\right) \quad \text{vs} \quad \text{softmax}(QK^T)$$

Without scaling, when$d_k = 64$, the inner product value can go up to± 64, so softmax is saturated. Scaling reduces it to the± 8range, resulting in a smooth distribution.

5.4 Scaled Dot-Product Attention avatar

Let’s start with the most basic form. Using PyTorch 2.0’s`F.scaled_dot_product_attention`(SDPA), it is memory efficient and fast.

```python
import torch
import torch.nn.functional as F

class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout: float = 0.0):
        super().__init__()
        self.dropout_p = dropout

    def forward(self, q, k, v, mask=None):
        # q, k, v: [batch, n_heads, seq, d_head]
        # mask: [batch, 1, seq, seq] or [seq, seq]
```

```
        out = F.scaled_dot_product_attention(
            q, k, v, attn_mask=mask,
            dropout_p=self.dropout_p if self.training else 0.0,
        )
        return out
```

SDPA is optimized at the C++ level, making it dozens of times faster than directly implementing softmax with a Python loop. It also automatically utilizes the latest algorithms such as Flash Attention.

Flash Attention

Flash Attention (Dao et al. 2022) is an algorithm that redesigns attention calculation to fit the GPU memory hierarchy, reducing memory access and improving speed by 2∼4 times. SDPA in PyTorch 2.0 automatically enables Flash Attention. We benefit from one line`F.scaled_dot_product_attention`without having to implement it ourselves.

5.5 causal mask (Causal Mask)

In an autoregressive language model, you should not look to the "future". This means that you should not look at token$t+1, t+2, \ldots$ when predicting token$t$. For this purpose, the upper triangle (above the main diagonal) of the attention score matrix is masked with$-\infty$so that it becomes 0 after softmax.

```
def make_causal_mask(seq_len: int, device="cpu") -> torch.Tensor:
    """Lower triangle mask. future tokens -infMasking with."""
    mask = torch.full((seq_len, seq_len), float("-inf"), device=device)
    mask = torch.triu(mask, diagonal=1)  # above the main diagonal -infas
    return mask
```

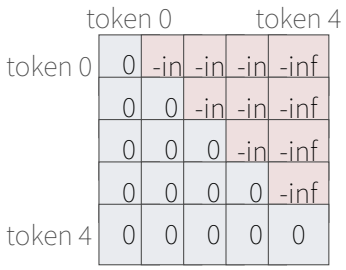


Figure 5.1: Causal Mask (5 – 5). blue = Allow attention (0), red = Block future tokens (– 1 –). line – 2 – is a token – 3 – heating – 4 – Indicates how much attention will be paid to the token.

When the mask is added to the attention score, the score of the masked location becomes$-\infty$, and after softmax, it becomes 0. In other words,$V$of future tokens will not contribute to the output.

5.5.1 why – impression?

In softmax's formula,softmax$(x_i) = e^{x_i} / \sum_j e^{x_j}$, if$x_i = -\infty$, it becomes$e^{-\infty} = 0$. Therefore, the weight of the masked position becomes exactly 0. This is different from simply adding 0: adding 0 leaves the original score intact, which is still positive after

softmaxing.

5.6 Multi-Head Attention

A single attention can only learn one "viewpoint". But there are many different relationships in language: grammatical relationships (subject-verb), semantic relationships (synonyms, antonyms), discourse relationships (denotative reference resolution), etc.**Multi-head attention (Multi-Head Attention)**divides it into multiple heads and processes it in parallel.

5.6.1 idea

The$d_model$dimension is divided into$h$heads, and each head independently performs attention in the$d_head = d_model/h$dimension. For example, if$d_model = 512, h = 8$then$d_head = 64$.

Multi-Head Attention

$$\text{MHA}(x) = \text{Concat}(\text{head}\_1, \ldots, \text{head}\_h)W\_O$$

$$\text{head}\_i = \text{Attention}(QW\_Q^i, KW\_K^i, VW\_V^i)$$

Concatenate the output of each head and linearly convert it toW_O.

5.6.2 Why split the head?

Single attention with$d_model = 512$and multihead attention with$d_head = 64, h = 8$have the same total number of parameters. However, in multi-head, each head can learn different patterns. for example:

- Head 1: Subject-verb relationship
- Head 2: Adjective-Noun Modification
- Head 3: Directive Reference
- Head 4: Sentence boundaries

If you actually visualize attention heads (e.g. BertViz), you can see that different heads pay attention to distinctly different patterns.

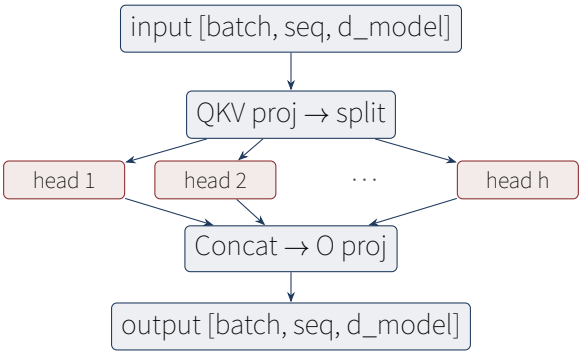


Figure 5.2: Multi-Head Attention structure. input – Split into two heads, perform parallel attention, and then merge.

5.6.3 avatar

For efficiency, calculateQ, K, Vat once. The input$x$is projected into the$3d_model$dimension and then divided by head.

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, config: ModelConfig):
        super().__init__()
        self.d_model = config.d_model
        self.n_heads = config.n_heads
        self.d_head = config.d_model // config.n_heads
        # Q, K, VCompute at once (combined projection)
        self.qkv_proj = nn.Linear(config.d_model, 3 * config.d_model, bias=config.use_bias)
        self.o_proj = nn.Linear(config.d_model, config.d_model, bias=config.use_bias)
        self.attention = ScaledDotProductAttention(dropout=config.dropout)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x, mask=None):
        batch, seq, _ = x.shape
        # [batch, seq, 3*d_model] -> [batch, seq, 3, n_heads, d_head]
        qkv = self.qkv_proj(x).view(batch, seq, 3, self.n_heads, self.d_head)
        # [3, batch, n_heads, seq, d_head]
        qkv = qkv.permute(2, 0, 3, 1, 4)
        q, k, v = qkv[0], qkv[1], qkv[2]

        if mask is not None and mask.dim() == 2:
            mask = mask.unsqueeze(0).unsqueeze(0)  # [1, 1, seq, seq]

        # Attention: [batch, n_heads, seq, d_head]
        attn_out = self.attention(q, k, v, mask=mask)
        # Merge Heads: [batch, seq, d_model]
        attn_out = attn_out.transpose(1, 2).contiguous().view(batch, seq, self.d_model)
        return self.dropout(self.o_proj(attn_out))
```

Why calculate QKV all at once?

The reason for using one`nn.Linear`with`3 * d_model`output instead of three separate`nn.Linear`is efficiency. GPUs are optimized for performing large matrix operations at once. One large operation is faster than three small operations. Partitioning is performed without memory reorganization with`view + permute`.

5.7 causality test

The most important test of the attention module is whether the causal mask actually blocks future tokens. The following test verifies this.

```python
def test_attention_is_causal(small_config):
    """location 의i 출력이 location i+Should not be affected after 1."""
    attn = MultiHeadAttention(small_config)
    attn.eval()
    x = torch.randn(1, 6, small_config.d_model)
    mask = make_causal_mask(6)
    out1 = attn(x, mask=mask)
    # xEven if you change the last position of , the output of the previous position should not change.
```

```
    x2 = x.clone()
    x2[:, -1] = torch.randn(1, small_config.d_model)
    out2 = attn(x2, mask=mask)
    # The first five positions must be identical
    assert torch.allclose(out1[:, :5], out2[:, :5], atol=1e-5)
```

If this test passes, you can be confident that the causal mask is working correctly. If it fails, the mask is upside down or the dimensions are wrong.

5.8 Self-Attention vs Cross-Attention

The implementation in this chapter is**Self attention (self-attention)**, whereQ, K, Vall come from the same sequence. The decoder in the translation model also uses**Cross attention (cross-attention)**, where$Q$comes from the decoder andK, Vcomes from the encoder. GPT is a decoder only structure, so it uses only self-attention.

5.8.1 How is cross-attention different??

In cross attention, they are$Q = x_decW_Q, K = x_encW_K$, and$V = x_encW_V$. Each position in the decoder refers to every position in the encoder output. In translation, this is the process of finding the corresponding word in the original text when the decoder generates the "sentence to be translated."

GPT does not have this cross attention. Predict the next token using only self-attention. This is why GPT is called a "decoder only" model.

5.9 Attention calculation volume analysis

The calculation amount of attention is$O(L^2 \cdot d)$.$L$is the sequence length. As the sequence becomes longer, it increases as a square.

sequence length	Attention matrix size	FP16 memories (single head)
512	512×512	0.5 MB
2048	2048×2048	8 MB
8192	8192×8192	128 MB
32768	32768×32768	2 GB

This is why processing long contexts is difficult. Volume 2, Chapter 7 covers how to reduce this cost with KV cache and PagedAttention.

5.10 practice: Attention weight visualization

By visualizing the attention weight of the trained model, you can see what patterns the model pays attention to.

```
import matplotlib.pyplot as plt
import math
from makellm.model.attention import make_causal_mask

# Manual calculation and visualization of attention weights (SDPAmanages the weights only internally, so calculate
```

```
    them directly when debugging)
model.eval()
with torch.no_grad():
    x = torch.randn(1, 10, model.config.d_model) # [batch=1, seq=10, d_model]
    x_norm = model.blocks[-1].ln1(x)
    qkv = model.blocks[-1].attn.qkv_proj(x_norm)
    batch, seq, _ = x.shape
    qkv = qkv.view(batch, seq, 3, model.config.n_heads, model.config.d_head).permute(2, 0, 3, 1, 4)
    q, k = qkv[0], qkv[1]  # [1, n_heads, seq, d_head]

    # Calculate the weight of the 0th head: (Q @ K^T) / sqrt(d_head)
    scores = (q[0, 0] @ k[0, 0].T) / math.sqrt(model.config.d_head)
    mask = make_causal_mask(seq)
    attn_weights = torch.softmax(scores + mask, dim=-1)  # [seq, seq]

plt.imshow(attn_weights.numpy(), cmap='viridis')
plt.colorbar()
plt.xlabel('key position')
plt.ylabel('query position')
plt.title('Attention weights (head 0, last layer)')
```

5.11 summation

- Attention is an operation that weights and adds values based on query-key similarity.
- Prevent softmax saturation by scaling to $\sqrt{d\_k}$.
- It becomes an autoregressive model by blocking future tokens with a causal mask.
- Multihead learns various relationships by dividing$d_model$into$h$heads.
- Maximize GPU efficiency by calculating QKV at once.
- By utilizing PyTorch 2.0's SDPA, optimized attention can be used in one line.
- Attention calculation amount is$O(L^2)$, which is a bottleneck in long sequences.

5.12 practice problems

1. $\sqrt{d_k}$How does learning change if scaling is removed? Calculate the distribution of softmax output when$d_k = 64$.
2. What happens if you don't put on a causal mask? Explain each during learning and inference.
3. If the number of heads$h$is set to 1 (single head), what is the difference from multi-head?
4. What happens if we make the number of heads$h$equal tod_model(i.e.$d\_head = 1$)?
5. Calculate how much the memory of attention matrix$L \times L$is when$L = 100,000$(based on FP16).

In the next chapter, we will assemble the transformer block including this attention. The

combination of attention + FFN + residual + LayerNorm creates a transformer.

Chapter 6

Assembling the transformer blocks

In this chapter,**Transformer block (transformer block)**is created by combining the attention from Chapter 5 and the embedding from Chapter 4. Attention alone is not enough; only a combination of FFN (Feed-Forward Network), residual connection, and layer normalization can learn meaningful expressions.

6.1 Structure of transformer block

One transformer block consists of four components:

1. **Multi-Head Attention**: Information exchange between tokens
2. **Feed-Forward Network (FFN)**: Nonlinear transformation by position
3. **Residual Connection**: Deep network training by adding input to output
4. **Layer Normalization**: Stabilization of learning

How these four things are combined is the key to the transformer block. Volume 1 uses the Pre-LN structure (GPT-2 style).

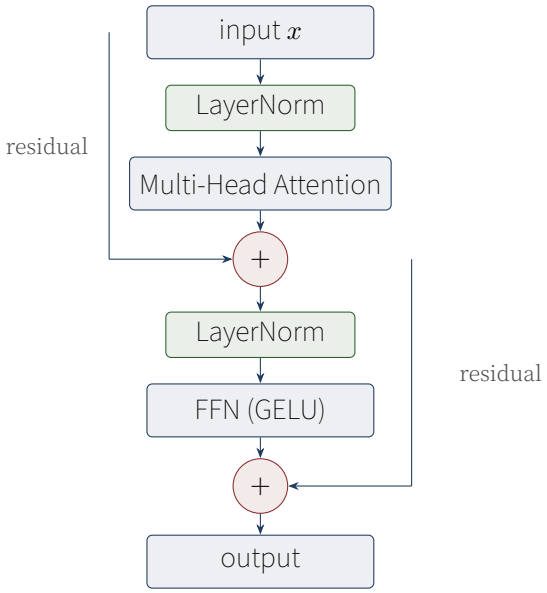


Figure 6.1: Pre-LN Transformer block structure. Attention and FFN Connect the residuals to each and LayerNormThere is.

6.2 Feed-Forward Network (FFN)

Attention performs "horizontal" information exchange between tokens. FFN, on the other hand, performs a "vertical" non-linear transformation at each token position. The structure is simple:

$$\text{FFN}(x) = \text{Linear\_2}(\text{GELU}(\text{Linear\_1}(x)))$$

Linear_1expands to$d_model \to d_ff$, andLinear_2contracts back to$d_ff \to d_model$.$d_ff$is usually$4d_model$, and this "expand-contract" pattern gives non-linear expressiveness.

6.2.1 intuition: why expand-Is it shrinking??

The reason for expanding$d_model = 512$to$d_ff = 2048$and then contracting it back to 512 is to "operate in a larger expression space." In higher-dimensional spaces, more complex decision boundaries can be drawn. Combined with the non-linearity of the activation function (GELU), FFN learns more than just a linear transformation.

In fact, research results show that FFN acts as a "key-value memory" (Geva et al. 2021). The interpretation is that the row vector ofLinear_1acts as the "key" and the column vector ofLinear_2acts as the "value", so FFN is actually an information retrieval mechanism.

```python
import torch.nn.functional as F

class FeedForward(nn.Module):
    def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ff)
        self.w2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)
        self._init_weights()

    def _init_weights(self) -> None:
        nn.init.normal_(self.w1.weight, mean=0.0, std=0.02)
        nn.init.normal_(self.w2.weight, mean=0.0, std=0.02)
        if self.w1.bias is not None:
            nn.init.zeros_(self.w1.bias)
        if self.w2.bias is not None:
            nn.init.zeros_(self.w2.bias)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """x: [batch, seq, d_model] -> [batch, seq, d_model]"""
        return self.dropout(self.w2(F.gelu(self.w1(x))))
```

6.3 GELU vs ReLU

The original Transformer used ReLU, but since GPT-2,**GELU(Gaussian Error Linear Unit)**is the standard. GELU uses a Gaussian cumulative distribution for smooth transitions instead of ReLU's "harsh" 0/1 threshold.

GELU

$$\text{GELU}(x) = x \cdot \Phi(x) \approx 0.5x \left(1 + \tanh\left[\sqrt{2/\pi}(x + 0.044715x^3)\right]\right)$$

$\Phi(x)$is the cumulative distribution function of the standard normal distribution.$x <$ 0also allows small negative values, so the gradient flow is better.

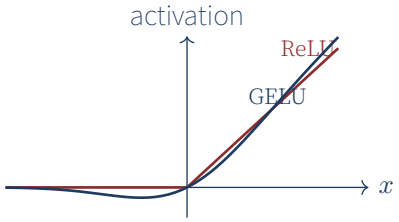


Figure 6.2: ReLU vs GELU. ReLUIs – is exactly 0, but, GELUIs 작은 음수값을 허용하여 부드럽게 전환.

6.3.1 why GELUIs it better??

ReLU becomes completely 0 at$x < 0$, so the gradient becomes 0 (“dead ReLU” problem). GELU allows small negative values so the gradient always flows. Additionally, the transition point is smooth and differentiable, making optimization stable.

In fact, most modern models, including GPT-2, BERT, and RoBERTa, use GELU. Volume 2 covers a more advanced activation function called SwiGLU.

6.4 Connect residuals (Residual Connection)

Concatenate residuals (residual connection)is an operation that adds input directly to output:$y = x + f(x)$. This simple idea made deep network learning possible. He et al. (2016) started with ResNet, which enabled learning over 100 layers in image classification.

6.4.1 intuition: Why residuals help?

If there are no residuals, the network must learn$f(x) = y$. The deeper you go, the more difficult it is to learn this mapping. If there are residuals, you only need to learn$f(x) = y - x$, that is, “residuals”. If the input$x$is already close toy, fonly needs a small correction.

Additionally, when backpropagating, the gradient flows directly through a “shortcut”. The derivative of$y = x + f(x)$is$\frac{dy}{dx} = 1 + f'(x)$. Even if it is$f'(x) = 0$, the gradient becomes 1 and does not disappear.

The transformer block has two residual connections: one around Attention and one around FFN. GPT has 12∼96 layers, so learning is impossible without residuals.

100-layer network without residual

Experimentally, if you subtract the residuals and train a 100-layer network, the loss does not decrease. This is because the gradient disappears during backpropagation and the front layer is not learned. Residual connections are simple but essential elements of deep networks.

6.5 Pre-LN vs Post-LN

There are two variations depending on the location of the LayerNorm.

6.5.1 Post-LN (Vaswani text)

The original Transformer placed the LayerNorm at Attention/FFNback:

$$x = \mathrm{LN}(x + \mathrm{Attn}(x))$$

However, this structure is unstable in learning in deep networks. It is very sensitive to initialization and prone to divergence without warmup.

6.5.2 Pre-LN (GPT-2 styles)

Place LayerNorm in Attention/FFNfront:

$$x = x + \mathrm{Attn}(\mathrm{LN}(x))$$

This structure has no LayerNorm in the residual path, so the gradient flows better. It is learned stably even in deep networks without warmup, making it the standard for modern LLM.

6.5.3 why Pre-LNIs this more stable??

After going through the residual path$x \to x + \mathrm{Attn}(x) \to \mathrm{LN}(\cdot)$in Post-LN, the gradient must pass through LayerNorm. LayerNorm is an operation that divides by variance, so it scales the gradient, but if this scaling is accumulated in a deep network, the gradient may disappear.

In Pre-LN,$x$flows directly from the residual path$x \to x+\mathrm{Attn}(\mathrm{LN}(x))$without going through LayerNorm. Only the attention output is added after going through LayerNorm. Learning is possible even in deep networks as the gradient flows directly through the identity path.

```
class TransformerBlock(nn.Module):
    def __init__(self, config: ModelConfig, layer_norm_style: str = "pre"):
        super().__init__()
        self.layer_norm_style = layer_norm_style
        self.ln1 = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
        self.ln2 = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
        self.attn = MultiHeadAttention(config)
        self.ffn = FeedForward(config.d_model, config.d_ff, config.dropout)

    def forward(self, x, mask=None):
        if self.layer_norm_style == "pre":
            # Pre-LN: normalization -> Attention -> residual
            x = x + self.attn(self.ln1(x), mask=mask)
            x = x + self.ffn(self.ln2(x))
        else:
            # Post-LN: Attention -> residual -> normalization
            x = self.ln1(x + self.attn(x, mask=mask))
```

```
            x = self.ln2(x + self.ffn(x))
        return x
```

Reason for using Pre-LN

Post-LN can disappear in deep networks because the initial gradient flows only through the LayerNorm and not the residual path. In Pre-LN, there is nothing in the residual path, so the gradient flows identity, making learning possible even with more than 100 layers. However, Pre-LN requires an additional LayerNorm at the end (implemented as `ln_f` in the next chapter).

6.6 Residual scaling

GPT-2 initializes the weights that directly contribute to the residual stream (W_O in attention, W_2 in FFN) to be smaller. This is to prevent the dispersion of residual output from increasing as the layer becomes deeper.

```
def _init_residuals(self, module: nn.Module) -> None:
    """Initialize projection weights that contribute directly to the residual stream to be small.."""
    if isinstance(module, nn.Linear):
        if module.weight.shape[0] == self.config.d_model:
            # Output to residual: Initialize to a smaller standard deviation
            nn.init.normal_(module.weight, mean=0.0,
                            std=0.02 / math.sqrt(2 * self.config.n_layers))
```

$1/\sqrt{2N}$ scaling corrects the fact that the variance of the residual output increases in proportion to the number of layers when there are N layers. The larger N, the smaller initialization is needed.

6.6.1 why – impression?

Each transformer block has two outputs (attention, FFN) that contribute to the residual. If there are N blocks, a total of $2N$ outputs are added to the residual. If the variance of each output is σ^2, the variance of the sum is $2N\sigma^2$. For this to become 1, it must be $\sigma = 1/\sqrt{2N}$.

6.7 block test

```
def test_block_shape(small_config):
    block = TransformerBlock(small_config)
    x = torch.randn(2, 8, small_config.d_model)
    out = block(x)
    assert out.shape == (2, 8, small_config.d_model)

def test_block_residual_connection(small_config):
    """If there is a residual connection, the difference between input and output must be finite."""
    block = TransformerBlock(small_config)
    block.eval()
    x = torch.randn(2, 8, small_config.d_model)
    out = block(x)
    diff = (out - x).abs().mean()
    assert diff.item() < 5.0  # Normalization problems if residuals are too large
```

```python

def test_feedforward_shape(small_config):
    ffn = FeedForward(small_config.d_model, small_config.d_ff)
    x = torch.randn(2, 8, small_config.d_model)
    out = ffn(x)
    assert out.shape == x.shape
```

6.8 Computation amount of transformer block

The FLOPs of one block are approximately:

- QKV projection:$3 \cdot L \cdot d^2$
- Attention Score:$L^2 \cdot d$
- Attention output projection:$L \cdot d^2$
- FFN: $2 \cdot L \cdot d \cdot d\_ff = 8Ld^2$ (since $d\_ff = 4d$)

The total is$O(L \cdot d^2)$, and$O(L^2 d)$of attention can be ignored when it is$L < d$. In actual GPT-3 ($d = 12288$,$L = 2048$), FFN is more dominant because it is$L \cdot d^2 = 3 \times 10^{11}andL^2 \cdot d = 5 \times 10^{10}$.

6.9 summation

- Transformer block = Attention + FFN + Residual connection + LayerNorm.
- FFN is a non-linear transformation by position (Linear→GELU→Linear).
- GELU improves learning stability with smoother transitions than ReLU.
- Residual connection is the core of deep network learning.
- Pre-LN has better learning stability than Post-LN, a modern LLM standard.
- Residual scaling$1/\sqrt{2N}$to prevent distributed explosions in deep networks.

6.10 practice problems

1. What happens if$d_ff$in FFN is changed to$d_model$instead of$4d_model$?
2. What phenomenon is observed when residual connections are removed and a 12-layer model is trained?
3. Why is an extra LayerNorm needed after the last block in Pre-LN?
4. What is the difference if I use Sigmoid Linear Unit (SiLU,$x \cdot \sigma(x)$) instead of GELU?
5. If we initialize a 100-layer model without residual scaling, what happens at the beginning of training?

In the next chapter, we build a complete GPT model by stacking these blocks$N$layers and build a learning loop.

Chapter 7

GPT Model completion and learning loop

In this chapter, we build a complete**GPT model**by stacking$N$layers of the transformer blocks from Chapter 6, and build a loop to learn this model. The dataset, loss function, optimizer, and scheduler are all implemented directly.

7.1 GPT model structure

GPT is an autoregressive language model with$N$layers of transformer decoder. The overall structure is as follows:

1. Token ID→[Token + Position Embedding]

Transformer Block $\times N$

Final LayerNorm

Head: $d\_model \to V$

2. logits

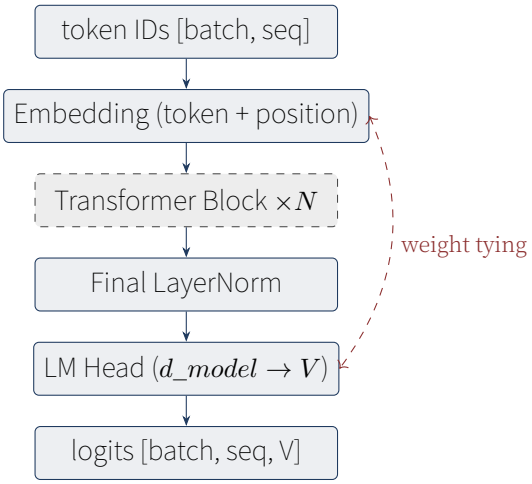


Figure 7.1: GPT model structure. Embedding weights and LM Head Share weights (weight tying) to save parameters.

7.2 weighted tie (Weight Tying)

The embedding matrix$E(V \times d_model)$and the LM Head matrix$W_out(d_model \times V)$actually contain the same information. The embedding of token$i$is$E[i]$, and$W_out^T[i]$is used when calculating logit. Sharing these two can save the number of parameters

by$V \times d_model$.

```
class GPT(nn.Module):
    def __init__(self, config: ModelConfig):
        super().__init__()
        self.config = config
        self.embedding = EmbeddingStack(
            vocab_size=config.vocab_size,
            d_model=config.d_model,
            max_len=config.context_length,
            pad_id=config.pad_token_id,
        )
        self.blocks = nn.ModuleList([
            TransformerBlock(config, layer_norm_style="pre")
            for _ in range(config.n_layers)
        ])
        self.ln_f = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
        self.lm_head = nn.Linear(config.d_model, config.vocab_size, bias=False)
        # weighted tie
        self.lm_head.weight = self.embedding.token_emb.embedding.weight
```

Advantages and disadvantages of weighted ties

merit: Parameter saving (about 1/3 reduction based on GPT-2 small), consistency of embedding and output.
disadvantage: Embedding also receives the gradient of the output layer, making learning more complicated. Some models (such as LLaMA) are trained separately without using ties.
GPT-2 and GPT-3 use ties. LLaMA and Mistral are not used. This book uses ties following the GPT style.

7.3 Forward Pass

```
    def forward(self, token_ids, return_hidden=False):
        batch, seq = token_ids.shape
        # Create a causal mask
        mask = make_causal_mask(seq, device=token_ids.device)
        # Embedding
        x = self.embedding(token_ids)
        # Transformer block pass
        for block in self.blocks:
            x = block(x, mask=mask)
        # Final normalization (Pre-LNso it is needed at the end)
        x = self.ln_f(x)
        # logit
        logits = self.lm_head(x)
        if return_hidden:
            return logits, x
        return logits
```

Why do you need LayerNorm at the end?

The Pre-LN block normalizes the input of each block. However, the output of the last block is added to the residual stream in an unnormalized state. This must be normalized before inserting it into the LM Head to ensure a stable output distribution. So`ln_f`(final LayerNorm) is needed.
In Post-LN, each block output is already normalized, so`ln_f`is unnecessary.

7.4 dataset: sliding windows

Language modeling datasets tokenize text and then split it into fixed-length chunks. In each chunk, input is the first$L-1$token, target is the last$L-1$token (shift one space).

```python
class LMDataset(Dataset):
    def __init__(self, text, tokenizer, context_length=128,
                 add_bos=True, add_eos=True):
        self.context_length = context_length
        # Tokenize text into long sequences
        if isinstance(text, str):
            text = [text]
        all_ids = []
        for t in text:
            ids = tokenizer.encode(t, add_bos=add_bos, add_eos=add_eos)
            all_ids.extend(ids)
        self._ids = all_ids
        # context_length+Split into 1 size chunks
        cl = context_length + 1
        self._chunks = []
        for i in range(0, len(all_ids), cl):
            chunk = all_ids[i:i+cl]
            if len(chunk) < cl:
                chunk = chunk + [tokenizer.special.pad_id] * (cl - len(chunk))
            self._chunks.append(chunk)

    def __getitem__(self, idx):
        chunk = self._chunks[idx]
        input_ids = torch.tensor(chunk[:-1], dtype=torch.long)
        target_ids = torch.tensor(chunk[1:], dtype=torch.long)
        return input_ids, target_ids
```

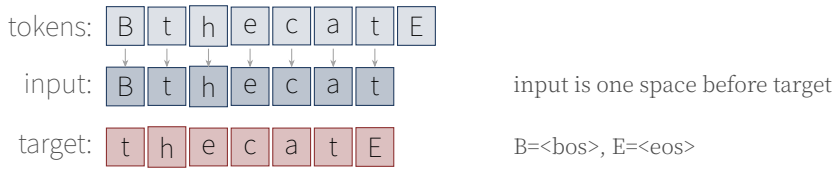


Figure 7.2: Sliding Window Dataset. inputclass targetis the same sequence one space shiftwhich. The model is inputlooking at targetpredict.

7.5 loss function: Cross-Entropy

The loss of the language model is**Cross entropy (cross-entropy)**. Measures the difference between the probability distribution predicted by the model and the correct answer (one-hot).

Cross-Entropy Loss

$$\mathcal{L} = -\frac{1}{N}\sum\_i = 1^N \log P(y\_i \mid x\_< i)$$

$y_i$is the correct answer token,$P$is the model's prediction probability. The lower the better.

```python
def cross_entropy_loss(logits, targets, ignore_index=-100):
    # logits: [batch, seq, V], targets: [batch, seq]
    batch, seq, vocab = logits.shape
    logits_flat = logits.view(-1, vocab)
    targets_flat = targets.view(-1)
    return F.cross_entropy(logits_flat, targets_flat, ignore_index=ignore_index)
```

7.5.1 Label Smoothing (select)

A technique that improves generalization performance by allowing some uncertainty instead of being 100

```python
def label_smoothing_loss(logits, targets, smoothing=0.1, ignore_index=-100):
    """1 for the correct answer token-smoothing probability, to the rest smoothing/vocab probability."""
    log_probs = F.log_softmax(logits.view(-1, logits.size(-1)), dim=-1)
    targets_flat = targets.view(-1)
    nll_loss = -log_probs.gather(1, targets_flat.clamp(min=0).unsqueeze(1)).squeeze(1)
    smooth_loss = -log_probs.mean(dim=-1)
    mask = (targets_flat != ignore_index).float()
    nll_loss = (nll_loss * mask).sum() / mask.sum().clamp(min=1)
    smooth_loss = (smooth_loss * mask).sum() / mask.sum().clamp(min=1)
    return (1.0 - smoothing) * nll_loss + smoothing * smooth_loss
```

7.6 optimizer: AdamW

AdamWis an optimization algorithm that combines Adam with weight decay. The key is not to apply weight decay to bias and LayerNorm.

```python
def build_optimizer(model, config):
    # Separate parameter groups: decay / no_decay
    decay, no_decay = [], []
    for name, p in model.named_parameters():
        if not p.requires_grad: continue
        if p.ndim < 2 or "bias" in name or "ln" in name.lower():
            no_decay.append(p)
        else:
            decay.append(p)
    return AdamW(
        [{"params": decay, "weight_decay": config.weight_decay},
```

```
        {"params": no_decay, "weight_decay": 0.0}],
        lr=config.learning_rate, betas=(0.9, 0.95)
    )
```

Why are bias and LayerNorm subtracted from weight decay?

Weight decay is a regularization that pulls the weights towards 0. Matrix weights have the effect of preventing overfitting, but the$\gamma, \beta$of bias and LayerNorm have low dimensions, so the effect of weight decay is minimal and may actually hinder learning. This is standard practice in the PyTorch community.

7.7 learning rate scheduler

At the beginning of learning, the learning rate is gradually warmed up and then decreased to a cosine curve. This**warmup + cosine**schedule is used as standard in GPT-3.

```
def build_scheduler(optimizer, config, total_steps):
    warmup = config.warmup_steps
    def lr_lambda(step):
        if step < warmup:
            return float(step) / max(1, warmup)
        progress = (step - warmup) / max(1, total_steps - warmup)
        if config.lr_scheduler == "cosine":
            return 0.5 * (1.0 + math.cos(math.pi * min(progress, 1.0)))
        return 1.0
    return LambdaLR(optimizer, lr_lambda)
```

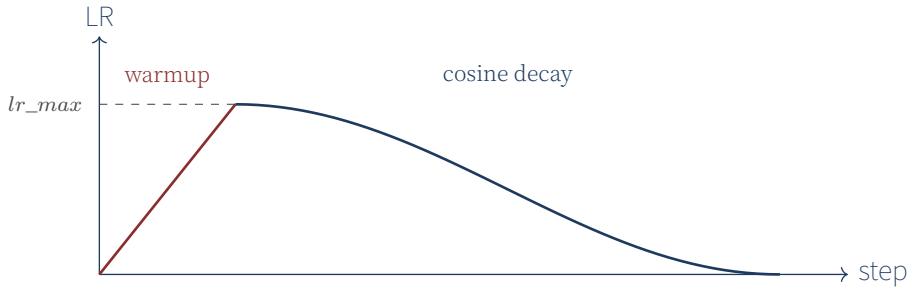


Figure 7.3: Warmup + Cosine Learning rate schedule. At first, raise it linearly., It then decreases to a cosine curve..

7.7.1 why warmupIs this necessary??

At the beginning of training, the weights are close to random. If a large learning rate is applied immediately, the gradient may become unstable and diverge. If you start with a small learning rate and increase it gradually, the model enters a stable region.

In particular, the initial gradient of attention can be very large (softmax saturation, etc.), and warmup alleviates this. Pre-LN structures are less sensitive to warm-up than post-LN, but are still safe to use.

7.8 gradient clipping

Limit the gradient norm to prevent the gradient from being too large and causing the weight to change significantly.

```
if config.grad_clip > 0:
    torch.nn.utils.clip_grad_norm_(model.parameters(), config.grad_clip)
```

Usually`grad_clip = 1.0`is used. If the gradient norm exceeds 1.0, it is scaled to 1.0.

7.9 trainer

Create a learning loop by tying all components together.

```
class Trainer:
    def __init__(self, model, train_dataset, config, model_config, eval_dataset=None):
        self.model = model
        self.config = config
        self.device = torch.device(config.device)
        self.model.to(self.device)
        self.train_loader = DataLoader(train_dataset, batch_size=config.batch_size,
                                       shuffle=True, collate_fn=collate_batch)
        # Calculate total number of steps
        total_steps = len(train_dataset) // config.batch_size * config.max_epochs
        self.optimizer = build_optimizer(model, config)
        self.scheduler = build_scheduler(self.optimizer, config, total_steps)
        # ...

    def train(self):
        self.model.train()
        for inputs, targets in self.train_loader:
            inputs, targets = inputs.to(self.device), targets.to(self.device)
            logits = self.model(inputs)
            loss = cross_entropy_loss(logits, targets,
                                      ignore_index=self.model_config.pad_token_id)
            self.optimizer.zero_grad(set_to_none=True)
            loss.backward()
            if self.config.grad_clip > 0:
                torch.nn.utils.clip_grad_norm_(self.model.parameters(),
                                               self.config.grad_clip)
            self.optimizer.step()
            self.scheduler.step()
```

7.10 Perplexity

Perplexity (Perplexity, PPL)is a representative evaluation index of language models.

Perplexity

$$\text{PPL} = \exp\left(\frac{1}{N}\sum\_i \log P(y\_i|x\_<i)\right) = \exp(\text{loss})$$

PPL indicates "how many candidates, on average, does the model wander through" when predicting the next token. PPL=1 means perfect, PPL=10 means you can narrow it down to 10 candidates.

PPL interpretation

- PPL = 1: Model is 100
- PPL = 10: Narrow down to one of 10 tokens. A level that is useful as a language model.
- PPL = 100: One of 100 tokens. Almost random.
- PPL = V (vocabulary size): Completely random. Not learned.

WikiText-103 PPL of GPT-2 small is about 17. GPT-3 175B is about 11.

7.11 Save checkpoint/load

Model weights can be periodically saved during training and resumed after interruption or used for inference.

```
def save_checkpoint(self, filename):
    path = self.out_dir / filename
    torch.save({
        "model_state": self.model.state_dict(),
        "optimizer_state": self.optimizer.state_dict(),
        "scheduler_state": self.scheduler.state_dict(),
        "global_step": self.global_step,
        "model_config": self.model_config.to_dict(),
        "trainer_config": self.config.to_dict(),
    }, path)

def load_checkpoint(self, path):
    ckpt = torch.load(path, map_location=self.device)
    self.model.load_state_dict(ckpt["model_state"])
    self.optimizer.load_state_dict(ckpt["optimizer_state"])
    self.scheduler.load_state_dict(ckpt["scheduler_state"])
    self.global_step = ckpt["global_step"]
```

7.12 summation

- GPT = Embedding + Transformer Block $\times N$ + LayerNorm + LM Head.
- Save parameters by sharing LM Head weights with embedding with weight ties.
- In Pre-LN structure, `ln_f` is required at the end.
- Language modeling creates (input, target) pairs with a sliding window.
- Loss is cross-entropy, optimizer is AdamW (bias/LayerNorm excludes weight decay).
- Warmup + Cosine learning rate schedule + gradient clipping is standard.

- Evaluate model quality with Perplexity =exp(loss).

7.13 practice problems

1. How much will the parameters increase in GPT-2 small (V=50257, d=768) if weight ties are not used?
2. How does learning change if the number of warmup steps is set to 0?
3. What is the effect of applying strong label smoothing to 0.3?
4. What can happen when learning without gradient clipping?
5. Can a model with a PPL of 1.0 actually exist? Why?

The next chapter covers how to generate text with the learned model. Greedy, Top-K, and Top-P sampling are all implemented.

Chapter 8

Text creation — Sampling Strategy and Decoding

In this chapter, we implement the process of generating text with the learned GPT model. It covers a variety of strategies, from simple greedy decoding to Top-K, Top-P, and Temperature sampling.

8.1 Principle of autoregressive generation

The creation of the language model is done in the**Autoregressive (autoregressive)**method. That is, it generates one token at a time, adds that token back to the input, and repeats the process to generate the next token.

1. Start with prompt$[w_1, \ldots, w_k]$.
2. Put it in the model to get the following token distribution$P(w_k+1 | w_1, \ldots, w_k)$.
3. Sample$w_k+1$from the distribution (or argmax).
4. Append to sequence:$[w\_1, \ldots, w\_k, w\_k+1]$.
5. 2∼4 iterations until the ending condition (<eos> or maximum length).

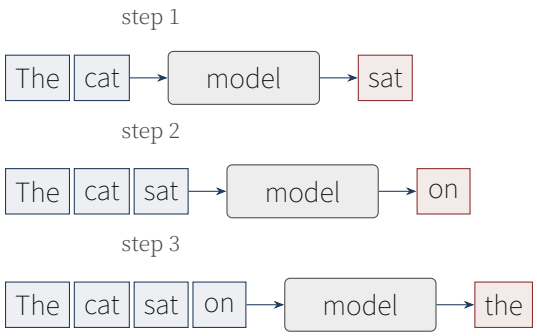


Figure 8.1: Autoregressive generation process. At each step, insert previous tokens into the model and predict the next token..

8.2 Greedy sampling

The simplest strategy is to select the token with the highest probability at each step.

```
class GreedySampler:
    def __call__(self, logits: torch.Tensor) -> torch.Tensor:
```

```
        # logits: [batch, vocab_size] -> token_ids: [batch]
        return torch.argmax(logits, dim=-1)
```

Greedy is deterministic and fast, but tends to select the same tokens every time, resulting in monotonous text. “The cat sat on the mat. The cat sat on the mat. It’s easy to fall into repetition like ”The cat…”.

Greedy’s Trap: Repeat Loop

Greedy decoding produces boring text by selecting only “safe” tokens. A more serious problem is that it falls into a repetitive loop. Same phenomenon as “I went to the store to buy some store to buy some store to buy…”.
Reason: If a loop is formed where the model predicts “store” followed by “to buy” with a high probability, and then “to buy” followed by “some store” with a high probability, greedy cannot get out of it.

8.3 Temperature sampling

Introduces a parameter that controls the “sharpness” of the distribution. Divide the logit by the temperature$T$and then softmax.

Temperature Sampling

$$P\_T(w) = \frac{\exp(\text{logit}(w)/T)}{\sum\_w' \exp(\text{logit}(w')/T)}$$

- $T < 1$: Distribution becomes sharper (closer to greedy)
- $T > 1$: Distribution becomes flatter (more random)
- $T = 1$: Original distribution

```
class TemperatureSampler:
    def __init__(self, temperature: float = 1.0):
        assert temperature > 0
        self.temperature = temperature

    def __call__(self, logits):
        probs = F.softmax(logits / self.temperature, dim=-1)
        return torch.multinomial(probs, num_samples=1).squeeze(-1)
```

8.4 Top-K sampling

To prevent tokens with low probability from being selected, only the top$K$tokens are considered.

```
class TopKSampler:
    def __init__(self, k: int = 50, temperature: float = 1.0):
        self.k = k
        self.temperature = temperature

```

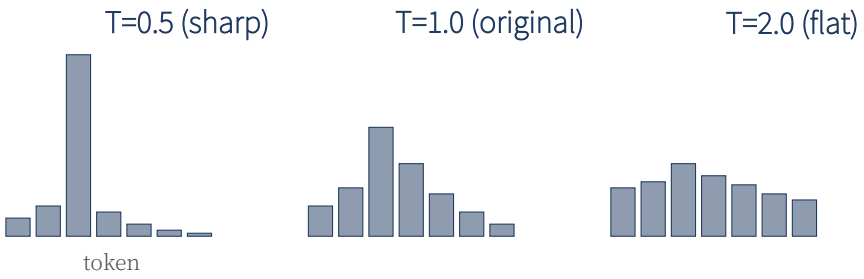


Figure 8.2: TemperatureDistribution changes according to. TIf is low, the distribution becomes sharp. greedygetting closer to, TThe higher it is, the flatter it becomes, increasing the randomness..

```
    def __call__(self, logits):
        scaled = logits / self.temperature
        top_values, top_indices = torch.topk(scaled, self.k, dim=-1)
        probs = F.softmax(top_values, dim=-1)
        sampled = torch.multinomial(probs, num_samples=1).squeeze(-1)
        return top_indices.gather(-1, sampled.unsqueeze(-1)).squeeze(-1)
```

8.4.1 Top-Kintuition

Cut off the “tails” (tokens with very low probability) from the logit distribution. For example, when the vocabulary is 50,000, considering only the top 50 means sampling from tokens with a probability mass of 99.99%.

If Top-K = 1, it is the same as greedy. If Top-K =V(vocabulary size), it is the same as temperature sampling.

8.5 Top-P (Nucleus) Sampling

Top-K fixes$K$, but Top-P dynamically adjusts the number of tokens considered depending on the distribution type. Only tokens until the cumulative probability reaches$P$are considered.

```
class TopPSampler:
    def __init__(self, p: float = 0.9, temperature: float = 1.0):
        self.p = p
        self.temperature = temperature

    def __call__(self, logits):
        scaled = logits / self.temperature
        probs = F.softmax(scaled, dim=-1)
        sorted_probs, sorted_indices = torch.sort(probs, descending=True, dim=-1)
        cumulative = torch.cumsum(sorted_probs, dim=-1)
        # PMasking from positions beyond
        sorted_mask = cumulative - sorted_probs > self.p
        sorted_probs = sorted_probs.masked_fill(sorted_mask, 0.0)
        sorted_probs = sorted_probs / sorted_probs.sum(dim=-1, keepdim=True).clamp(min=1e-10)
        sampled = torch.multinomial(sorted_probs, num_samples=1).squeeze(-1)
        return sorted_indices.gather(-1, sampled.unsqueeze(-1)).squeeze(-1)
```

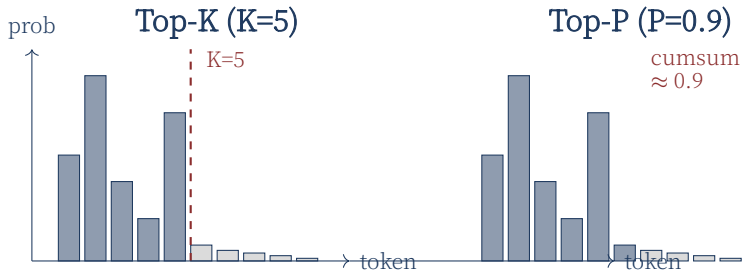


Figure 8.3: Top-K vs Top-P. Top-Kis a fixed number, Top-Pis the cumulative probability PSelect until.

8.5.1 Top-K vs Top-P

	Top-K	Top-P (Nucleus)
Number of tokens considered	fixed (K)	dynamic (depending on distribution)
When distribution is sharp	consider too many tokens	only consider few tokens (automatic)
When distribution is flat	consider too few	consider many tokens (automatic)
Adaptability	Low	High

Top-P usually produces more natural results. This is because the model considers only a few tokens when it is confident, and many tokens when it is uncertain.

8.6 Sampling Strategy Comparison

strategy	manifold	consistency	example
Greedy	Low	High	Code Generation, Math
Temperature (T=0.7)	Medium	Medium	General use
Top-K (K=50)	Medium	High	Chatbot
Top-P (P=0.9)	Medium	High	Chatbot, Creation
Top-P (P=0.95)	High	Medium	Creative Writing

8.7 Implementing the creation function

Create a`generate`function that combines all samplers.

```
@torch.no_grad()
def generate(model, tokenizer, prompt, max_new_tokens=50,
             sampler=None, device="cpu", stop_on_eos=True):
    if sampler is None:
        sampler = GreedySampler()
    model.eval()
    model.to(device)
    ids = tokenizer.encode(prompt, add_bos=True, add_eos=False)
```

```
    input_ids = torch.tensor([ids], dtype=torch.long, device=device)
    eos_id = tokenizer.special.eos_id
    context_length = model.config.context_length

    for _ in range(max_new_tokens):
        # Context length limit
        x = input_ids[:, -context_length:] if input_ids.shape[1] > context_length else input_ids
        logits = model(x)
        next_logits = logits[:, -1, :]  # last location
        next_id = sampler(next_logits)
        input_ids = torch.cat([input_ids, next_id.unsqueeze(0)], dim=1)
        if stop_on_eos and next_id.item() == eos_id:
            break
    return tokenizer.decode(input_ids[0].tolist())
```

8.8 Context length limit

The model can only process up to`context_length`specified during training. During construction, if the sequence exceeds this length, the oldest token is truncated (sliding window). It's an approximation, but it works. Volume 2 covers RoPE's extrapolation technique to overcome this limitation.

Problem with Sliding Window

If you cut off the context, the model will forget the "front part". "Once upon a time, there was a princess named Aurora. She lived in a castle. One day, [Long explanation] ... If you cut out the first part of 'Aurora decided to", the context that "Aurora" is the princess's name is lost.
Solution: Use a longer context model (RoPE extrapolation), or compress it by summarizing it. Covered in Volume 2.

8.9 sampler test

```
def test_greedy_picks_max():
    sampler = GreedySampler()
    logits = torch.tensor([[1.0, 3.0, 2.0, 0.5]])
    out = sampler(logits)
    assert out.item() == 1  # largest value

def test_top_k_limits_choices():
    torch.manual_seed(0)
    logits = torch.tensor([[1.0, 5.0, 4.0, 3.0, 0.1]])
    sampler = TopKSampler(k=2, temperature=1.0)
    for _ in range(20):
        idx = sampler(logits).item()
        assert idx in (1, 2)  # always top-one of 2

def test_top_p_limits_choices():
    torch.manual_seed(0)
    logits = torch.tensor([[0.0, 10.0, 9.0, 0.0, 0.0]])
    sampler = TopPSampler(p=0.5, temperature=1.0)
```

```
    for _ in range(20):
        idx = sampler(logits).item()
        assert idx == 1  # Only the highest probability tokens
```

8.10 practice: Comparison of various sampling

scripts/tiny_train.pyRunning the demo allows you to compare the output of five different sampling strategies.

```
cd make-llm-basic
python scripts/tiny_train.py --epochs 5 --vocab 500
```

Example output:

```
prompt: 'the'
  [Greedy           ] the cat sat on the mat
  [Temperature=0.5  ] the lazy dog sleeps
  [Temperature=1.0  ] the brown fox jumps
  [Top-K=10         ] the quick fox runs
  [Top-P=0.9        ] the warm sun shines
```

8.11 Advanced decoding techniques (introduction)

Although not implemented in this book, we introduce advanced techniques used in real systems:

8.11.1 Beam Search

At each step,Kcandidate sequences are maintained, and the sequence with the highest score is selected at the end. Suitable for translation, summarization, etc., but not suitable for chatbots (lack of variety).

8.11.2 Speculative Decoding

The smaller "draft" model generates$K$tokens first, then the larger model verifies it in parallel. If correct, adopt$K$at once, if incorrect, start from the beginning. Throughput 2∼can be improved by 3 times.

8.11.3 Contrastive Search

Modern techniques for maintaining consistency while avoiding repetition. Select various tokens by penalizing them for similarity to previous token representations.

8.12 summation

- Autoregressive generation generates one token at a time and extends the sequence.
- Greedy: The token with the highest probability. Dangers of monotony and repetitive loops.
- Temperature: Controls the sharpness of the distribution. The lower T, the more

deterministic.

- Top-K: Only the top K items are considered. Simple but non-adaptive.
- Top-P: Only tokens up to cumulative probability P are considered. Adaptive to distribution.
- Truncates the oldest token if the context length is exceeded.
- Advanced techniques such as speculative decoding and beam search also exist.

8.13 practice problems

1. Analyze why greedy decoding falls into an iterative loop. Under what kind of distribution do loops occur most often?
2. How is it different from greedy when Temperature approaches 0? What happens if it is 0?
3. What is the effect of applying Top-K and Top-P at the same time? (i.e. filtering Top-P among Top-K candidates)
4. What sampling is best for chatbots? Explain why.
5. Why is speculative decoding fast?

This completes the core implementation of Volume 1. The next chapter summarizes the entire book and introduces advanced topics that will be covered in Volume 2.

Chapter 9

Clean up and next steps

Congratulations! If you have followed this far, you have now implemented all components of LLM, from tokenizer to text generation. This chapter summarizes Volume 1 and introduces advanced topics to be covered in Volume 2,Make LLM-advanced.

9.1 Volume 1 Summary

9.1.1 implemented

page	Implementation details
Chapter 3	BPE, Unigram Tokenizer (Learning/Encoding/Decoding)
Chapter 4	Token/Positional Embedding, EmbeddingStack
Chapter 5	Scaled Dot-Product Attention, Causal Mask, Multi-Head Attention
Chapter 6	FeedForward (GELU), Transformer Block (Pre-LN)
Chapter 7	GPT Model, LMDataset, Cross-Entropy, AdamW, Cosine LR, Trainer
Chapter 8	Greedy/Temperature/Top-K/Top-P Sampler, generate function

9.1.2 test

All 48 unit tests pass. We verified that all modules operate correctly through geometric tests, numerical tests, and integration tests.

```
$ pytest tests/ -v
======================== 48 passed in 2.47s ========================
```

9.1.3 What the reader has

Now the reader can:

- Learn tokenizer with your own corpus
- Create a GPT model of arbitrary size (just change the config)
- Run training loop on CPU/GPU
- Generate text with various sampling strategies
- Understand the inner workings of HuggingFace Transformers when reading their code

9.2 Limitations of Volume 1

To be honest, the model created in Volume 1 is far from ChatGPT. There are three main limitations:

size: Volume 1 model has 10 million~100 million parameters. GPT-3 has an estimated parameter of 175 billion, and GPT-4 has an estimated parameter of 1.7 trillion. Distributed learning is essential to overcome this size difference.

architecture: Volume 1 is close to the original Transformers from 2017. Modern LLM (LLaMA, Mistral, GPT-4) uses more efficient and stable components such as RoPE, GQA, SwiGLU, and RMSNorm.

array: Volume 1 model simply learned "next token prediction". To create a chatbot that answers user questions, alignment techniques such as SFT (Supervised Fine-Tuning), RLHF (Reinforcement Learning from Human Feedback), and DPO (Direct Preference Optimization) are required.

9.3 Covered in Volume 2

Volume 2Make LLM-advanceddeals with how to overcome the three limitations above.

9.3.1 Distributed Learning (2 – Chapter 4)

- **DDP (Distributed Data Parallel)**: Replicate learning of the same model on multiple GPUs.
- **FSDP (Fully Sharded Data Parallel)**: Save memory by splitting the model into shards.
- **ZeRO-1/2/3**: Shading optimizer/gradient/parameters.
- **Pipeline Parallel**: Place models on different GPUs for each layer.
- **Tensor Parallel**: Splitting a single layer across multiple GPUs (Megatron-LM).

9.3.2 Modern Architecture (5 – Chapter 6)

- **RoPE (Rotary Position Embedding)**: Position encoding with complex rotation. Excellent extrapolation.
- **GQA (Grouped Query Attention)**: Save inference memory by reducing KV head.
- **SwiGLU**: Improved FFN performance with GLU variant. Used by LLaMA.
- **RMSNorm**: A lightweight version of LayerNorm.
- **MoE (Mixture-of-Experts)**: Only the top K tokens for each token are activated in the expert network pool.

9.3.3 Quantization and inference optimization (Chapter 7)

- **INT8/INT4 Quantization**: Saves memory 4~8 times by converting weights to low-bit integers.
- **AWQ**: Minimizing accuracy loss using activation value statistics.
- **GPTQ**: Quantization using secondary information.

- **KV Cache**: Accelerate inference by reusing K and V of previous tokens.
- **PagedAttention**: Address memory fragmentation with page-level KV management in vLLM.

9.3.4 Alignment and fine tuning (Chapter 8)

- **SFT (Supervised Fine-Tuning)**: Learning chatbot behavior through supervised learning.
- **RLHF**: Optimizing human preferences with compensation model + PPO.
- **DPO**: Direct preference optimization without the complexity of RLHF.
- **LoRA**: Efficient fine tuning with low-rank adapter.

9.3.5 Data Pipeline (Chapter 9)

- **quality filter**: Filter by length, language, and repetition rate.
- **MinHash Deduplication**: Duplicate document detection using Jaccard similarity approximation.
- **synthetic data**: Template-based learning data generation.

9.4 Recommended Learning Path

Recommended activities before moving on to Volume 2:

1. **small project**: Study texts in your field of interest (e.g. Wikipedia Korean, GitHub code) with Volume 1 code. About 100,000 tokens can be learned in 1 hour.
2. **read the paper**: Read "Attention Is All You Need" (Vaswani et al. 2017) and "Language Models are Few-Shot Learners" (Brown et al. 2020). Having read Volume 1, you will understand much better.
3. **HuggingFace read code**: Open `transformers/models/gpt2/modeling_gpt2.py`. Compare how similar it is to the code we created and what differences there are.

9.5 finally

LLM may seem complicated, but it is ultimately a combination of tokenizer, embedding, attention, FFN, and learning loop. If you learn these basics, you can understand any new architecture or new optimization technique. In Volume 2, we will build ChatGPT level technology on this foundation.

Never stop coding. Never stop experimenting. That's the only way to improve your skills.

End of Volume 1

dirmich

Appendix A

Math Basics Supplement

This appendix provides supplementary explanations of the mathematical concepts used in the main text. This is reference material for readers who know calculus and basic linear algebra but are not familiar with matrix differentiation.

A.1 vectors and matrices

vectoris a one-dimensional array of numbers. Expressed as$\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$.**procession**is a two-dimensional array.$A \in \mathbb{R}^{m \times n}ismrown$column.

- **dot product (dot product)**:$\mathbf{a} \cdot \mathbf{b} = \sum\_i a\_i b\_i$. Returns a scalar.
- **matrix multiplication**:$C = AB$where$C_{ij} = \sum_k A_{ik} B_{kj}$.
- **transposition(transpose)**:A^Tchanges rows and columns.
- **norm**:$\|\mathbf{x}\|\_2 = \sqrt{\sum\_i x\_i^2}$. The length of the vector.

A.2 Softmax

Softmaxis a function that converts a random real vector into a probability distribution.

Softmax

$$\text{softmax}(x\_i) = \frac{e^{x\_i}}{\sum\_j e^{x\_j}}$$

All elements of the output are between 0 and 1, and the sum is 1. Used for attention weight, classification model output, etc.

A.3 Cross-Entropy

Cross entropy (cross-entropy)measures the difference between two probability distributionsp, q.

Cross-Entropy

$$H(p, q) = -\sum\_i p\_i \log q\_i$$

In classification problems,$p$is the answer one-hot vector, and$q$is the model prediction softmax output. Simplify the single correct answer$y$to$H = -\log q_y$.

A.4 chain rule and Backpropagation

chain rule(chain rule)is the differentiation method for composite functions.

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

Since a neural network is a series of composite functions, the chain rule can be used to calculate the derivative from input to output. This is the mathematical basis of**backpropagation**.

PyTorch automates this process, so we do not need to derive the differential equation ourselves. However, to understand "which parameters affect loss and how much", you must know the concept of the chain rule.

A.5 matrix differentiation

When the loss$\mathcal{L}$depends on the matrix$W$,$\partial\mathcal{L}/\partial W$is a matrix of the same shape as$W$, and each element is$\partial\mathcal{L}/\partial W_ij$.

Jacobianis a matrix representing the differentiation of a vector function. When$\mathbf{y} = f(\mathbf{x})J_ij = \partial y_i/\partial x_j$.

For more information, see "The Matrix Cookbook" (Petersen & Pedersen).

A.6 normal distribution

Probability density function of**Normal distribution (normal distribution)**$\mathcal{N}(\mu, \sigma^2)$:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Often used for weight initialization. Using$\mu = 0, \sigma = 1/\sqrt{d}$ensures that the output variance remains constant even as the input dimension increases (the basic idea of Xavier initialization).

A.7 Information Theory Basics

Entropy (entropy)measures the "randomness" of a probability distribution.

$$H(p) = -\sum\_ip\_i \log p\_i$$

KL diffusion(KL divergence)is the difference between the two distributions:

$$D\_KL(p\|q) = \sum\_ip\_i \log \frac{p\_i}{q\_i}$$

Cross-entropy is decomposed into$H(p)+D_KL(p\|q)$. Since$H(p)$is fixed, minimizing cross-entropy is the same as minimizingD_KL.

A.8 Optimization Basics

Gradient descent (gradient descent): Update parameters to reduce loss.

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

$\eta$is the learning rate.**Adam**is an optimization algorithm that combines momentum and adaptive learning rate.**AdamW**is a version that combines Adam with weight decay and is the standard for LLM learning.

Detailed derivation can be found in Goodfellow et al. “Deep Learning” 4∼Please refer to Chapter 8.

Appendix B

Full list of codes

This appendix collects the complete code of the core modules implemented in Volume 1. Although the text is partially excerpted for explanation purposes, the complete code is provided here. Check the GitHub repository for the latest version.

B.1 project structure

```
make-llm-basic/
  src/makellm/
    __init__.py
    tokenizer/
      __init__.py
      base.py        # Tokenizer abstract class
      bpe.py         # BPE tokenizer
      unigram.py     # Unigram tokenizer
    model/
      __init__.py
      embedding.py   # TokenEmbedding, PositionalEmbedding
      attention.py   # MultiHeadAttention, CausalMask
      transformer_block.py
      gpt.py         # GPT model
    training/
      __init__.py
      dataset.py     # LMDataset
      loss.py        # cross_entropy_loss
      optimizers.py  # AdamW + cosine scheduler
      trainer.py     # Trainer class
    inference/
      __init__.py
      sampler.py     # Greedy, Top-K, Top-P, Temperature
      generate.py    # generate function
    utils/
      __init__.py
      config.py      # ModelConfig, TrainerConfig
      seed.py        # set_seed
      logging.py     # Logger
  tests/             # pytest unit testing
  book/              # of this book LaTeX sauce
```

B.2 run test

```
# install package
cd make-llm-basic
pip install -e .

# Run full test
pytest tests/ -v

# Test only specific modules
pytest tests/test_tokenizer.py -v
pytest tests/test_model.py -v
pytest tests/test_training.py -v
```

B.3 Quickstart example

```
"""Using all modules from Volume 1 end-to-end example."""
from makellm.tokenizer import BPETokenizer
from makellm.model import GPT
from makellm.training import LMDataset, Trainer
from makellm.inference import generate, TopPSampler
from makellm.utils import ModelConfig, TrainerConfig, set_seed

set_seed(42)

# 1. Tokenizer training
corpus = ["the quick brown fox jumps", "the lazy dog sleeps"] * 100
tokenizer = BPETokenizer()
tokenizer.train(corpus, vocab_size=500)

# 2. Model creation
model_config = ModelConfig(
    vocab_size=tokenizer.vocab_size,
    context_length=32, d_model=64, n_heads=4, n_layers=2, d_ff=128
)
model = GPT(model_config)
print(f"Model: {model}")
print(f"Parameters: {model.num_parameters():,}")

# 3. Dataset preparation
dataset = LMDataset(corpus, tokenizer, context_length=32)

# 4. learning
trainer_config = TrainerConfig(
    batch_size=4, learning_rate=3e-4, max_steps=100,
    warmup_steps=10, device="cpu"
)
trainer = Trainer(model, dataset, trainer_config, model_config)
trainer.train()

# 5. Text creation
sampler = TopPSampler(p=0.9, temperature=0.8)
text = generate(model, tokenizer, prompt="the", max_new_tokens=20, sampler=sampler)
print(f"Generated: {text}")
```

B.4 By module API summation

module	main API
makellm.tokenizer	BPETokenizer, UnigramTokenizer
makellm.model	GPT, MultiHeadAttention, TransformerBlock
makellm.training	LMDataset, Trainer, cross_entropy_loss
makellm.inference	generate, GreedySampler, TopKSampler, TopPSampler
makellm.utils	ModelConfig, TrainerConfig, set_seed

B.5 Setting data class

```python
@dataclass
class ModelConfig:
    vocab_size: int = 1000
    context_length: int = 128
    d_model: int = 128
    n_heads: int = 4
    n_layers: int = 4
    d_ff: int = 512
    dropout: float = 0.1
    use_bias: bool = True
    layer_norm_eps: float = 1e-5
    pad_token_id: int = 0

@dataclass
class TrainerConfig:
    batch_size: int = 32
    learning_rate: float = 3e-4
    weight_decay: float = 0.1
    max_epochs: int = 10
    warmup_steps: int = 100
    lr_scheduler: str = "cosine"
    grad_clip: float = 1.0
    log_every: int = 10
    eval_every: int = 200
    save_every: int = 1000
    seed: int = 42
    device: str = "cpu"
    out_dir: str = "./runs"
```

By adjusting these settings, you can experiment with models of different sizes. For example, a similar setup to GPT-2 small is`d_model=768, n_heads=12, n_layers=12, context_length=1024`(but actual training requires significant computing resources).

B.6 license

The code is released under the MIT license. It can be freely copied, modified, and used commercially. The text of the book is CC BY-NC-SA 4.0 (author verification required).

Appendix C

Frequently Asked Questions and Pitfalls

This appendix summarizes problems and questions frequently encountered by readers while studying Volume 1. Each item deals with specific situations that may occur during actual implementation.

C.1 environmental related

C.1.1 Q: PyTorch After installation CUDAis not recognized

A: Check two things. First, make sure you have the NVIDIA drivers installed (run`nvidia-smi`). Second, whether the PyTorch version and CUDA version are compatible. For example, if you install PyTorch for CUDA 12.1 on a CUDA 11.8 system, it will not be recognized.

```
# CUDA Check version
nvidia-smi | grep "CUDA Version"

# PyTorch Reinstall (CUDA 12.For 1)
pip uninstall torch
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

C.1.2 Q: Learning is too slow

A: Check the following:

1. GPU is being used (set`device="cuda"`).
2. Is the batch size too small (low GPU utilization)?
3. Is data loading the bottleneck (increases`num_workers`).
4. Are you using mixed precision (BF16)?

C.2 Tokenizer related

C.2.1 Q: BPEThe tokenizer learned with “hi”It tokenizes strangely.

A: Because Korean was not sufficiently included in the corpus. BPE is frequency-based, so rare characters are broken down into bytes. way out:

- Increase the Korean corpus ratio.
- Increase vocabulary size (about 20,000 Korean syllables + subwords).
- Use a professional tokenizer like SentencePiece.

C.2.2 Q: encode/decode Round trip fails

A: Check space handling. The "□" character in BPE may not be converted to a space when decoding. In our implementation, the`_postprocess`method handles this:

```
def _postprocess(self, text):
    text = text.replace("□", " ")
    return text.strip()
```

C.2.3 Q: special token IDWhat happens if changes

A: Fatal. The model was trained assuming`<pad>=0`, `<bos>=1`, `<eos>=2`, `<unk>=3`. If the special token ID changes while retraining the tokenizer, the model weights are no longer valid. Always fix special token IDs.

C.3 model related

C.3.1 Q: Attention output NaNThis comes out

A: Three causes are common:

1. **Softmax saturation**: Logit is too large, so softmax is saturated at 0/1. Check scaling ($\sqrt{d\_k}$).
2. **Learning rate too large**: Gradient explosion. Try reducing the learning rate to 1/10.
3. **FP16 overflow**: The activation value exceeds the FP16 range (± 65504). Switch to BF16.

```
# NaN check
if torch.isnan(loss):
    print("NaN detected! Check learning rate and gradients.")
    print(f"Max grad: {max(p.grad.abs().max() for p in model.parameters() if p.grad is not None)}")
```

C.3.2 Q: The model repeats the same word

A: Check your sampling strategy. Greedy is prone to falling into repetitive loops. Switch to Top-P (P=0.9) or set Temperature to 0.7~0.9. Adding a repetition penalty is also effective.

```
# Repetition penalty (simplified)
def apply_repetition_penalty(logits, generated_ids, penalty=1.2):
    for token_id in set(generated_ids.tolist()):
        logits[token_id] /= penalty
    return logits
```

C.3.3 Q: An error occurs if the context length is exceeded.

A: If input exceeding`context_length`of the model is entered, an error occurs at the embedding stage. Notice the cut to the sliding window in the`generate`function:

```
if input_ids.shape[1] > context_length:
    x = input_ids[:, -context_length:]  # Keep only the most recent
```

C.4 learning related

C.4.1 Q: Losses do not decrease

A: Check the following:

1. The learning rate is either too small (1e-6 or less) or too large (1e-2 or more).
2. Too little data (at least a few thousand tokens required).
3. Bad optimizer (check AdamW's beta, eps).
4. Gradient does not flow (check requires_grad).

```
# Check gradient flow
for name, p in model.named_parameters():
    if p.requires_grad and p.grad is None:
        print(f"No gradient for {name}")
    elif p.grad is not None and p.grad.abs().max() < 1e-10:
        print(f"Vanishing gradient for {name}")
```

C.4.2 Q: Loss occurs in the early stages of learning.

A: Possibility of insufficient warmup or incorrect initialization. Try this:

- Increase Warmup Steps (100∼1000).
- Check for gradient clipping (`grad_clip=1.0`).
- Reduce initialization standard deviation (0.02→0.01).

C.4.3 Q: Loading a checkpoint gives different results

A: Two causes:

1. **Seed not set**: Check whether`set_seed(42)`has been called.
2. **dropout**: Check whether the model has been switched to`eval()`mode.
3. **CuDNN indeterminism**:`torch.backends.cudnn.deterministic = True`.

C.5 Distributed Learning (Preview of Volume 2)

C.5.1 Q: DDPIf you learn with , the speed is rather slow.

A: Possible communication bottleneck. Check the following:

- The batch size is too small, so the computation/communication ratio is low.
- Use slower communications (PCIe, etc.) rather than NVLink.
- Gradient synchronization occurs at every step (reduced to gradient accumulation).

C.5.2 Q: FSDPat OOMThis is me

A: Not enough activation memory. Turn on checkpointing or reduce batch size. CPU offload is also considered.

C.6 Debugging Tips

C.6.1 Verify step by step

Complex bugs are broken down into small units and verified. Testing only the tokenizer, testing only the attention, etc.

```
# Tokenizer standalone test
tok = BPETokenizer()
tok.train(["hello world"], vocab_size=50)
print(tok.encode("hello"))
print(tok.decode(tok.encode("hello")))

# Attention stand-alone test
attn = MultiHeadAttention(config)
x = torch.randn(1, 4, 32)
out = attn(x)
print(out.shape, out.dtype, torch.isnan(out).any())
```

C.6.2 shapeAlways print

80

```
print(f"input shape: {x.shape}")
print(f"after embedding: {emb.shape}")
print(f"after attention: {attn_out.shape}")
```

C.6.3 Start with small input

Start with batch 1, sequence 4, check operation, then increase size. Debugging on large inputs can be confusing because there is too much output.

C.7 Performance Optimization Tips

C.7.1 Use mixed precision

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()
with autocast(dtype=torch.bfloat16):
    logits = model(inputs)
    loss = criterion(logits, targets)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

Unlike FP16, BF16 does not require a scaler. Recommended for A100 and above.

C.7.2 torch.compile

Compiling a model with`torch.compile`in PyTorch 2.0 results in a 2x speedup from 1.5~.

```
model = GPT(config)
model = torch.compile(model)  # compilation
```

C.7.3 profiling

```
with torch.profiler.profile(activities=[torch.profiler.ProfilerActivity.CPU,
                                        torch.profiler.ProfilerActivity.CUDA]) as prof:
    # learning code
prof.export_chrome_trace("trace.json")
```

Open with `chrome://tracing` in Chrome browser to analyze bottleneck.

Appendix D

Practice Problem Solutions

This appendix provides answers to the text exercises. Please try it yourself and then check.

D.1 Chapter 1 Answers

1-1. Rate ChatGPT's answer.

Best answer: "Language models are trained to predict next tokens, but with sufficiently large models and diverse data, they implicitly learn question-answer patterns. When the user inputs a question in the form of a question, the model generates the answer pattern that follows the question in the training data."

1-2. "The sun sets in the east..." Next word probability.

For humans, "soaring" is rated at over 99%. Reasons: (1) everyday observation, (2) linguistic collocation ("rising from the east" is a fixed expression), (3) strong constraints of context.

1-3. N-gram sparsity problem.

For N-gram to predict the next word "polar bear dance", the next word "polar bear dance" must have appeared in the training data. If this 3-gram has not been learned, the probability becomes 0, and general words such as "more" are recommended through smoothing.

1-4. Why transformers are better than RNNs for parallel processing.

RNN requires sequential processing because the calculation of point t depends on the hidden state of point $t-1$. Transformers process all positions simultaneously with attention, making them optimal for GPU parallelization.

1-5. Chinchilla reference data scale.

If the parameters are increased by 10 times, the data must also be increased by about 10 times (maintaining a 20 token/parameter ratio). In other words, a 10x model requires 200x data.

D.2 Chapter 3 Answers

3-1. What if we merge less frequent pairs?

Patterns that do not appear frequently become tokens, making the vocabulary inefficient. When rare pairs like “xz” are merged, they result in tokens that are rarely used in actual text.

3-2. Unigram penalty changed to -5.0.

Since the penalty for substrings not in the vocabulary is small, Viterbi attempts segmentation more actively. As a result, the tendency to break down into many smaller tokens.

3-3. Korean BPE, vocabulary 100 vs 1000.

Vocabulary 100: Break down “hello” into tokens of 5 or more bytes. Very inefficient. Vocabulary 1000: Able to learn subwords such as “hello” and “hello”. Significantly shortened sequence length.

3-4. Viterbi length reduced to upper limit of 4.

Tokens longer than 5 characters disappear. Words like “hello” are broken down into “he”, “ll”, and “o”, increasing the sequence length.

The need for **3-5.** <bos>.

The model will work without it, but if you explicitly mark the beginning of a sequence, the model will recognize that “a new sequence starts from here”. Also useful for distinguishing between multiple sequences during batch processing.

D.3 Chapter 4 Answers

4-1. When the embedding dimension is too small or large.

If it is small (8): It is difficult to distinguish between words, making it impossible to learn semantic similarity. Larger (4096): risk of parameter explosion, running out of memory, and overfitting. Usually 256∼4096 used.

4-2. Sinusoidal constant $100 \to 100$.

As the cycle becomes shorter, it becomes difficult to distinguish even close locations. Lowered the “resolution” of location encoding.

4-3. Scaling omitted.

Location embeddings dominate token embeddings, causing the model to rely solely on location. Loss of semantic information.

4-4. Learnable location embeddings, max_len exceeded.

An error occurred because the index was out of range. A method that allows extrapolation, such as RoPE, is needed.

4-5. Change LayerNorm ϵ value.

If ϵ is large, normalization becomes weak and output variance increases. If it is too small, the denominator will be close to 0, causing numerical instability.

D.4 Chapter 5 Answers

5-1.softmax when scaling is omitted.

When$d_k = 64$, the inner product value can go up to± 64, so softmax is saturated in one-hot. Gradient 0, learning stops.

5-2. Causal mask not used.

During training: the model "sees" future tokens and answers correctly, resulting in lower learning loss but no generalization. During inference: There are no future tokens, so a learning-inference gap occurs. Performance plummets.

5-3. Number of heads 1 (single head).

Learn only one attention pattern. Decreased performance because grammar, semantics, and discourse relationships must all be processed by one head.

5-4. Number of heads =$d\_model (d\_head = 1)$.

Each head processes only one dimension. The meaning of attention disappears and becomes a simple weighted sum. Practically the same as a linear transformation.

5-5. Attention matrix memory,$L = 100,000$.

$100000^2 \times 2$bytes (FP16) = 20 GB. Based on single head. 32 heads = 640 GB. Virtually impossible.

D.5 Chapter 6 Answers

6-1. $d\_ff = d\_model$.

FFN's expansion-contraction pattern disappears, reducing non-linear expressiveness. Similar to a deeper linear transformation.

6-2.12th floor with no residuals.

When going backwards, learning is not possible because the gradient does not reach the previous layer. Losses are not reduced.

6-3. Necessity of`ln_f`in Pre-LN.

Pre-LN normalizes each block input, but leaves the last block output unnormalized. Normalization is required before inserting into the LM Head.

6-4. SiLU instead of GELU.

Minor performance difference. SiLU ($x \cdot \sigma(x)$) also has a smooth transition. Some models (FFN in LLaMA is SwiGLU but uses SiLU as activation) are adopted.

6-5.100 layers without residual scaling.

The initial activation value distribution accumulates for each layer and explodes. NaN is likely to occur in the early stages of training.

D.6 Chapter 7 Answers

7-1. No weighted ties used, GPT-2 small.

Add$V \times d = 50257 \times 768 \approx 38.6M$parameter. Total 124M→163M.

7-2. Warmup 0.

The gradient at the beginning of training is unstable, so there is a possibility of divergence. Especially serious in Post-LN. Pre-LN is less sensitive but still recommended.

7-3. Label smoothing 0.3.

Strong smoothing makes the model less confident about the correct answer. Generalization improves, but learning loss remains high. Usually 0.1 is appropriate.

7-4. No gradient clipping.

When a gradient explodes, the weights change significantly and the model collapses. Especially dangerous at large learning rates.

7-5. Is PPL 1.0 possible?

Theoretically impossible. Natural language has inherent uncertainty, so the next token cannot be 100

D.7 Chapter 8 Answers

8-1. Causes Greedy repeating loops.

If a loop is formed where the distribution predicts "to buy" highly after "store" and "some store" after "to buy" highly, greedy cannot get out of it. This is because only the path with the highest probability is followed.

8-2. Temperature $\to 0$.

If$T = 0$, softmax becomes argmax and is the same as greedy. If you are$T = 0.01$, you are almost greedy. The distribution is so sharp that the diversity is 0.

8-3. Top-K and then Top-P.

After reducing the candidates to K using Top-K, select up to the cumulative probability P among them. More powerful filtering. Quality improves but variety decreases.

8-4. Sampling suitable for chatbots.

Top-P (P=0.9, T=0.7∼0.9). Balance between variety and consistency. Greedy is monotonous, pure sampling lacks consistency.

8-5. Why speculative decoding is fast.

If the small model quickly generates K tokens, the large model verifies the K tokens at once (parallel processing). If correct, K are adopted in one step, so the number of steps in the large model is reduced to 1/K.

Appendix E

Advanced topic preview

This appendix briefly introduces the advanced topics covered in Volume 2,Make LLM-advanced. It helps readers who have completed Volume 1 decide on the next direction of study.

E.1 distributed learning

Volume 1 model was trained on a single GPU/CPU. However, models larger than 70B do not fit into single GPU memory.

DDP (Distributed Data Parallel): Place a model replica on each GPU and split only the data. The simplest.

FSDP (Fully Sharded Data Parallel): Shading all parameters, gradients, and optimizer states. 70B+ models can be trained.

ZeRO: DeepSpeed's staged sharding. Progressive memory savings with ZeRO-1/2/3.

pipeline parallel: Split the model into GPU by layer. Suitable for inter-node communication.

tensor parallel: Split the weight of a single layer. Megatron-LM method.

E.2 modern architecture

Volume 1 is close to the original Transformers from 2017. Modern LLMs use more efficient components.

RoPE (Rotary Position Embedding): Position encoding with complex rotation. Excellent extrapolation makes it advantageous for processing long contexts. LLaMA, using Mistral.

GQA (Grouped Query Attention): Save inference memory by reducing KV head. LLaMA-2 70B uses GQA-8.

SwiGLU: Improved FFN performance by activating GLU variants.

RMSNorm: A lightweight version of LayerNorm. Calculation amount reduced by 10~20%.

MoE (Mixture-of-Experts): Among multiple experts, only the top K are activated for each token. Mixtral 8x7B operating with 47B total parameters and 13B active.

E.3 Quantization and inference optimization

INT8/INT4 Quantization: Saves memory 4∼8 times by converting weights to low-bit integers. Critically reduce inference costs.

AWQ: Preserve important channels with activation statistics. High accuracy even with INT4.

KV Cache: Reuse previous K, V in autoregressive generation. Every step complexity$O(L^2) \to O(L)$.

PagedAttention: Page-level KV management in vLLM. Multi-sequence throughput 2∼4x improvement.

E.4 Alignment and fine tuning

Volume 1 model only "predicts the next token". Sorting is required to create a chatbot.

SFT (Supervised Fine-Tuning): Supervised learning with command-response pairs. Learning basic chatbot behavior.

RLHF: Preference optimization with compensation model + PPO. Used by ChatGPT.

DPO (Direct Preference Optimization): Direct preference optimization without compensation model. Simple and effective. Zephyr, using Llama-2-Chat.

LoRA: Save 99

E.5 data pipeline

quality filter: Remove low-quality text by length, language, and repetition rate.

MinHash: Duplicate document detection using Jaccard similarity approximation. 30∼50

synthetic data: Generate domain-specific data with powerful models. Evol-Instruct, Self-Instruct, etc.

E.6 Volume 2 Study Preparation

Volume 2 expands on the code of Volume 1. Take the GPT model from Volume 1:

- Wrapped in distributed learning (2∼Chapter 4)
- Replace attention with GQA (Chapter 5)
- Changing FFN to SwiGLU (Chapter 5)
- Add LoRA adapter (Chapter 8).

If you are familiar with Volume 1, you can naturally follow Volume 2. It is recommended that you experiment by modifying the Volume 1 code yourself.

E.7 Recommended Learning Path

1. Volume 1 Learning small models with code (`scripts/tiny_train.py`).

2. Experiment with your own corpora (Wikipedia Korean, GitHub code, etc.).
3. Move on to volume 2 and build a distributed learning environment.
4. Fine tuning experiment with LoRA (`scripts/lora_finetune.py`).
5. Optimize inference with quantization (`scripts/quantize_demo.py`).
6. Practice serving with real HuggingFace models (vLLM, TGI).

By following this path, you will have the ability to understand and run an LLM at ChatGPT level. Never stop coding. Never stop experimenting. That's the only way to improve your skills.